Universidad de Granada
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Grado en Ingeniería Informática

Dirección y Gestión de Proyectos (DGP)
Curso Académico 2026–2027



Documento de Equipo Iteracion 1

05/11/2026

Proyecto AUTOVIDA: Sistema de Agenda y Asignación de Tareas Accesible para Alumnado NEAE / PTVAL

En colaboración con el C.E.E. Fundación Purísima Concepción de Granada

Equipo de Trabajo:
CLENCH Software Development



Asignación de Roles:

Miguel Ángel Luque	Catalogador
Yeray Rodríguez Navas	Gestor de Calidad
Julián Carrión Tovar	Gestor Accesibilidad
Pablo Hernández Ibáñez	Presentador
Pablo de la Torre Roldán	Moderador
José Rodríguez Fernández	Coordinador
Amine Azzammouri	Coordinador (Refuerzo)

Índice

1. Planificación temporal 3
1.1. Planificación inicial Iteración 1 . . . 3
1.2. Planificación final Iteración 1 (real) . . . 4
1.3. Comparativa . . . 5

2. Documentos generados durante el desarrollo 6
2.1. Documentos generados en el backend . . . 6
2.1.1. Diagrama Entidad Relación de la Base de Datos . . . 6
2.1.2. Guía para iniciar el backend . . . 7
2.1.3. Guía de uso del backend . . . 14
2.1.4. Guía de errores del backend . . . 31
2.2. Documentos generados en el frontend . . . 35
2.2.1. Flujo de pantallas del frontend . . . 35
2.2.2. Pantallas del frontend . . . 35
2.2.3. Guía del frontend . . . 39
2.3. Documentos comunes al frontend y backend . . . 61
2.3.1. Guía para lanzar el sistema (backend + frontend) . . . 61

3. Enlaces a código (GIT) y JIRA 69
3.1. Repositorio de código . . . 69
3.2. Gestión del proyecto en JIRA . . . 69

4. Actas de Equipo 70
4.1. Acta 07: Diseño y arquitectura . . . 71
4.1.1. Asistencia . . . 71
4.1.2. Orden del día y desarrollo . . . 71
4.1.3. Acuerdos . . . 71
4.1.4. Compromisos . . . 72
4.1.5. Próxima convocatoria . . . 72
4.2. Acta 08: Revisión de documentación . . . 73
4.2.1. Asistencia . . . 73
4.2.2. Orden del día y desarrollo . . . 73
4.2.3. Acuerdos . . . 73
4.2.4. Compromisos . . . 74
4.2.5. Próxima convocatoria . . . 74
4.3. Acta 09: Desarrollo e integración de frontend y backend . . . 75
4.3.1. Asistencia . . . 75
4.3.2. Orden del día y desarrollo . . . 75
4.3.3. Acuerdos . . . 75
4.3.4. Compromisos . . . 76
4.3.5. Próxima convocatoria . . . 76
4.3.6. Anotaciones . . . 76
4.4. Acta 10: Seguimiento del desarrollo y cierre de la iteración 1 . . . 77

4.4.1. Asistencia . . . 77
4.4.2. Orden del día y desarrollo . . . 77
4.4.3. Acuerdos . . . 77
4.4.4. Compromisos . . . 78
4.4.5. Próxima convocatoria . . . 78
4.5. Acta 11: Revisión final de la primera iteración . . . 79
4.5.1. Asistencia . . . 79
4.5.2. Orden del día y desarrollo . . . 79
4.5.3. Acuerdos . . . 79
4.5.4. Compromisos . . . 80
4.5.5. Próxima convocatoria . . . 80

5. Registro de horas del equipo **81**

1. Planificación temporal

1.1. Planificación inicial Iteración 1

La planificación inicial recoge las historias de usuario previstas para la primera iteración, organizada en torno a la gestión de usuarios, la relación entre alumnos y tutores, y la administración de tareas y clases. Esta propuesta establece el alcance de partida del sprint, previsto entre el 26 de octubre y el 16 de noviembre.

SCRUM Sprint 1 26 oct – 16 nov (31 actividades)

SCRUM-8	Como administrador quiero crear usuarios
SCRUM-10	Como administrador quiero eliminar los datos de los usuarios, para retirar del sistema a aquellos que no van a utilizar el sistema
SCRUM-11	Como administrador, quiero consultar las cuentas de usuario, para gestionar los usuarios y sus roles.
SCRUM-9	Como administrador quiero modificar los datos de los usuarios, para mantener actualizada su información y sus credenciales
SCRUM-12	Como usuario, quiero iniciar sesión con mis credenciales
SCRUM-13	Como usuario, quiero poder cerrar sesión.
SCRUM-14	Como administrador, quiero asociar alumnos con sus tutores y familiares
SCRUM-23	Como administrador, quiero desasociar un alumno de un tutor o familiar
SCRUM-16	Como administrador, quiero consultar los alumnos asociados a un tutor
SCRUM-20	Como tutor responsable de un alumno, quiero consultar los alumnos que tengo asignados
SCRUM-21	Como miembro personal del centro, quiero consultar los alumnos que tengo asignados para supervisarlos en función del aula
SCRUM-26	Como tutor, quiero poder crear y asignar una tarea a un alumno
SCRUM-33	Como personal, quiero poder crear y asignar una tarea a un alumno
SCRUM-31	Como tutor, quiero eliminar una tarea de la agenda del alumno
SCRUM-41	Como personal, quiero eliminar una tarea de la agenda del alumno
SCRUM-322	Como administrador quiero crear clases
SCRUM-323	Como administrador quiero eliminar clases
SCRUM-324	Como administrador quiero modificar clases
SCRUM-326	Como personal quiero consultar las clases que tengo asignado
SCRUM-327	Como personal, quiero apuntar un alumno a mi clase
SCRUM-328	Como personal, quiero eliminar un alumno de mi clase
SCRUM-331	Administrador asigna profesor a clase
SCRUM-321	Como personal, quiero crear una tarea para una clase entera
SCRUM-336	Como personal, quiero eliminar una tarea de una clase entera
SCRUM-338	Como personal, quiero modificar las tareas de una clase entera
SCRUM-337	Como personal, quiero consultar las tareas de una clase entera

1.2. Planificación final Iteración 1 (real)

La planificación final refleja el alcance real realizado de la iteración.

SCRUM Sprint 1 26 oct – 16 nov (31 actividades)

SCRUM-8	Como administrador quiero crear usuarios
SCRUM-10	Como administrador quiero eliminar los datos de los usuarios, para retirar del sistema a aquellos que no van a utilizar el sistema
SCRUM-11	Como administrador, quiero consultar las cuentas de usuario, para gestionar los usuarios y sus roles.
SCRUM-9	Como administrador quiero modificar los datos de los usuarios, para mantener actualizada su información y sus credenciales
SCRUM-12	Como usuario, quiero iniciar sesión con mis credenciales
SCRUM-13	Como usuario, quiero poder cerrar sesión.
SCRUM-14	Como administrador, quiero asociar alumnos con sus tutores y familiares
SCRUM-23	Como administrador, quiero desasociar un alumno de un tutor o familiar
SCRUM-16	Como administrador, quiero consultar los alumnos asociados a un tutor
SCRUM-20	Como tutor responsable de un alumno, quiero consultar los alumnos que tengo asignados
SCRUM-21	Como miembro personal del centro, quiero consultar los alumnos que tengo asignados para supervisarlos en función del aula
SCRUM-26	Como tutor, quiero poder crear y asignar una tarea a un alumno
SCRUM-33	Como personal, quiero poder crear y asignar una tarea a un alumno
SCRUM-31	Como tutor, quiero eliminar una tarea de la agenda del alumno
SCRUM-41	Como personal, quiero eliminar una tarea de la agenda del alumno
SCRUM-333	como alumno, quiero eliminar mis propias tareas
SCRUM-322	Como administrador quiero crear clases
SCRUM-323	Como administrador quiero eliminar clases
SCRUM-324	Como administrador quiero modificar clases
SCRUM-326	Como personal quiero consultar las clases que tengo asignado
SCRUM-327	Como personal, quiero apuntar un alumno a mi clase
SCRUM-328	Como personal, quiero eliminar un alumno de mi clase
SCRUM-331	Administrador asigna profesor a clase
SCRUM-27	Como tutor, quiero consultar las tareas de la agenda de un alumno por fecha
SCRUM-36	Como personal del centro, quiero consultar las tareas de la agenda de un alumno por fecha
SCRUM-43	Como alumno, quiero poder crear mis propias tareas en mi agenda personal
SCRUM-329	Como administrador, quiero cambiar el avatar de un alumno para que pueda reconocer su perfil
SCRUM-321	Como personal, quiero crear una tarea para una clase entera
SCRUM-336	Como personal, quiero eliminar una tarea de una clase entera
SCRUM-338	Como personal, quiero modificar las tareas de una clase entera
SCRUM-337	Como personal, quiero consultar las tareas de una clase entera

1.3. Comparativa

En la planificación realizada al inicio de la iteración se optó por implementar una cantidad reducida de tareas debido a los múltiples cambios pendientes de realizar en la propuesta técnica entregada inicialmente, lo que restó tiempo efectivo al equipo para empezar con el desarrollo. En vista del rápido avance del equipo se propuso la idea de añadir las siguientes historias de usuario (SCRUM) pertenecientes a la iteración 2:

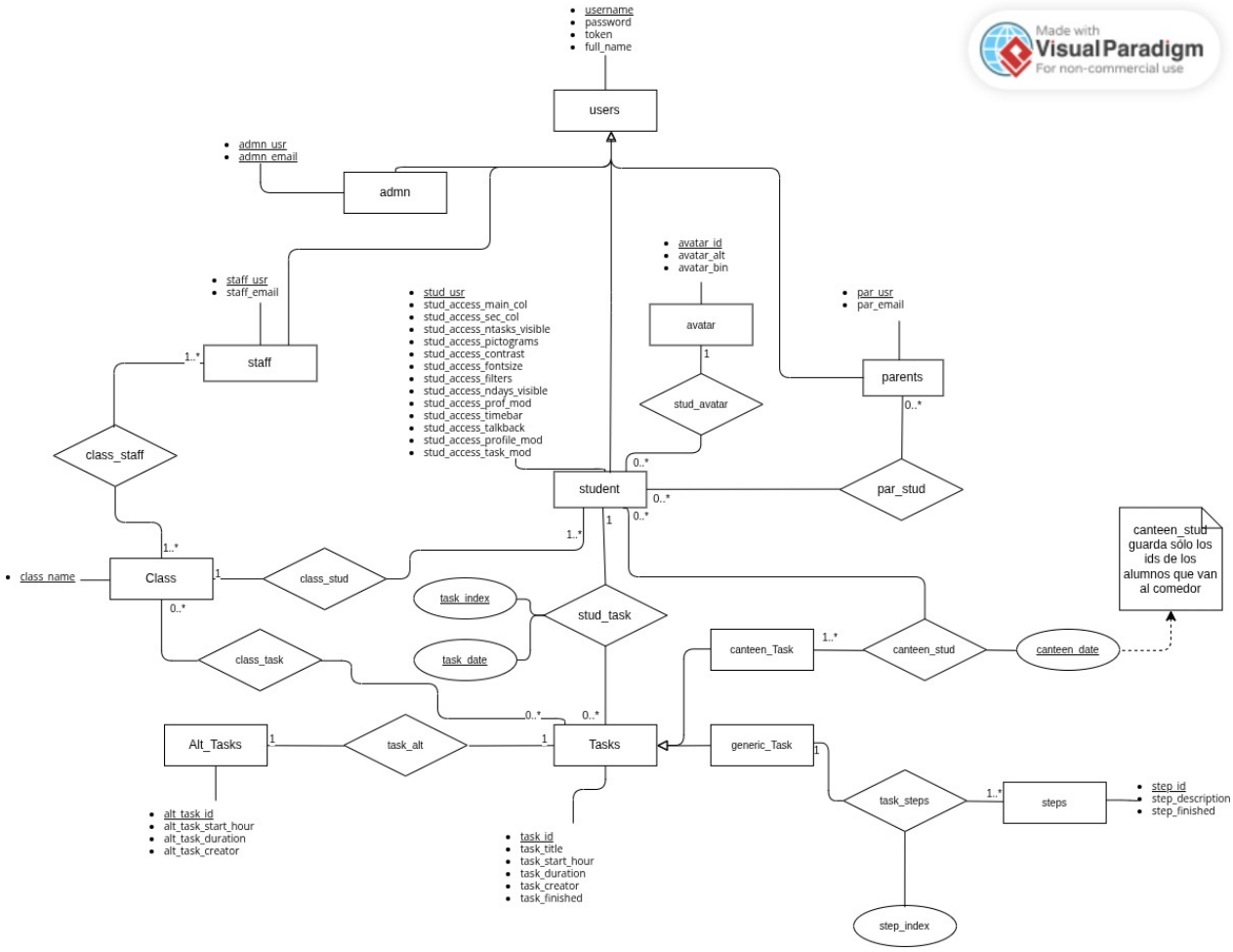
1. SCRUM 27: Como tutor, quiero consultar las tareas de la agenda de un alumno por fecha.
2. SCRUM 36: Como personal del centro, quiero consultar las tareas de la agenda de un alumno por fecha.
3. SCRUM 43: Como alumno, quiero poder crear mis propias tareas en mi agenda personal.
4. SCRUM 329: Como administrador, quiero cambiar el avatar de un alumno para que pueda reconocer su perfil.
5. SCRUM 333: Como alumno, quiero eliminar mis propias tareas.

El objetivo perseguido con esta ampliación es el de implementar al completo la épica de la creación y asignación de Tareas a alumnos. Adicionalmente, con estas modificaciones creemos que se podrá reducir la carga de trabajo de la siguiente iteración, ya que consideramos que requerirá mayores tiempos de desarrollo.

2. Documentos generados durante el desarrollo

2.1. Documentos generados en el backend

2.1.1. Diagrama Entidad Relación de la Base de Datos



2.1.2. Guía para iniciar el backend

Índice

- Instalación de herramientas 7
- Pasos para desplegar la base de datos en tu PC 7
- Pasos seguidos para desarrollar la base de datos por primera vez 8
- Resumen de comandos 12
- Resultado esperado 13

INSTALACIÓN DE HERRAMIENTAS

Para desarrollar el proyecto necesitaremos instalar las siguientes herramientas:

- **Flutter y Android Studio:** desarrollo de la aplicación móvil.
- **FastAPI (Python):** desarrollo del backend y de la API.
- **PostgreSQL:** sistema de gestión de bases de datos.
- **Docker:** ejecución de PostgreSQL en un contenedor.

PASOS PARA DESPLEGAR LA BASE DE DATOS EN TU PC

1. Descargar e instalar uv

Ejecutamos el siguiente comando en la terminal para descargar e instalar uv, el gestor de paquetes y proyectos de Python:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

2. Reiniciar la terminal

Cerramos y volvemos a abrir la terminal para que los cambios surtan efecto y el sistema reconozca el comando uv.

3. Editamos archivo .env en la carpeta

```
DB_USER=autovida_admin
DB_PASSWORD=<contraseña acordada>
DB_HOST=localhost
DB_PORT=5432
DB_NAME=autovida_db
```

4. Iniciar la base de datos

Nos situamos en la carpeta del proyecto FastAPI, donde se encuentra el archivo `docker-compose.yml`, y ejecutamos:

```
sudo docker compose up -d
```

Este comando inicia los servicios definidos en Docker Compose en segundo plano.

5. Iniciar FastAPI

Para iniciar el servidor de desarrollo de FastAPI, ejecutamos:

```
uv run fastapi dev main.py
```

PASOS SEGUIDOS POR YERAY PARA DESARROLLAR LA BASE DE DATOS POR PRIMERA VEZ

1. FastAPI

1.1. Instalar uv

Copiamos y pegamos el siguiente comando en la terminal para descargar e instalar uv:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

1.2. Crear el proyecto e instalar FastAPI

Creamos el proyecto con uv y accedemos a su directorio:

```
uv init mda-dgp-26/27 --bare
cd mda-dgp-26/27
```

A continuación, añadimos FastAPI con sus dependencias estándar:

```
uv add "fastapi[standard]"
```

1.3. Instalar las dependencias necesarias

Instalamos las dependencias para trabajar con PostgreSQL, gestionar los modelos de datos y realizar migraciones de la base de datos:

```
uv add "fastapi[standard]" sqlalchemy psycopg2-binary alembic pydantic-settings
```

Añadimos las dependencias necesarias para la seguridad y la autenticación:

```
uv add "passlib[bcrypt]" pyjwt
```

- **Passlib:** permite gestionar el hashing de contraseñas.

- **PyJWT:** permite trabajar con JSON Web Tokens (JWT) para la autenticación.

Por último, añadimos el controlador moderno de PostgreSQL para Python:

```
uv add "psycopg[binary]"
```

> **Nota:** esta última dependencia se añade porque puede ser necesaria para utilizar la versión moderna del controlador de PostgreSQL (`psycopg`), especialmente si el proyecto utiliza SQLAlchemy con este controlador.

---

2. PostgreSQL

2.1. Seleccionar la versión de PostgreSQL

Vamos a utilizar **PostgreSQL 18**, una versión estable que ofrece las funcionalidades necesarias para el proyecto.

Además, utilizaremos la variante **Alpine**, que ocupa menos espacio que otras imágenes de Docker y permite reducir el tamaño del contenedor.

La imagen elegida es:

```
postgres:18-alpine
```

2.2. Crear el archivo `docker-compose.yml`

En el directorio raíz del proyecto, creamos un archivo llamado `docker-compose.yml` con el siguiente contenido:

```yaml
services:
  db:
    image: postgres:18-alpine
    container_name: autovida_postgres
    restart: always
    environment:
      POSTGRES_USER: autovida_admin
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: autovida_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql

volumes:
  postgres_data:
```

Descripción de la configuración:

Parámetro	Descripción
`image`	Imagen de PostgreSQL 18 basada en Alpine.
`container_name`	Nombre del contenedor: `autovida_postgres`.
`restart`	Reinicia automáticamente el contenedor cuando corresponde.
`POSTGRES_USER`	Usuario de la base de datos.
`POSTGRES_PASSWORD`	Contraseña del usuario, obtenida de una variable de entorno.
`POSTGRES_DB`	Nombre de la base de datos que se creará inicialmente.
`ports`	Expone el puerto 5432 del contenedor en el puerto 5432 del equipo.
`volumes`	Permite conservar los datos de PostgreSQL.

Importante: antes de iniciar el contenedor, configura la variable `POSTGRES_PASSWORD` en un archivo `.env` situado en el mismo directorio:

```
POSTGRES_PASSWORD=tu_contraseña_segura
```

Sustituye `tu_contraseña_segura` por una contraseña propia y no compartas el archivo `.env`.

> **Nota sobre el almacenamiento:** en PostgreSQL 18, la ruta de datos recomendada para la imagen oficial es `/var/lib/postgresql`. Por eso se utiliza esta ruta como punto de montaje del volumen.

2.3. Iniciar PostgreSQL

Desde el directorio que contiene `docker-compose.yml`, ejecutamos:

```
docker compose up -d
```

Si tu sistema necesita permisos de administrador para ejecutar Docker, puedes utilizar:

```
sudo docker compose up -d
```

Este comando crea e inicia el contenedor de PostgreSQL en segundo plano.

Para comprobar que el contenedor está funcionando, ejecutamos:

```
docker ps
```

Debería aparecer el contenedor `autovida_postgres` en la lista.

---

3. Conectar FastAPI con PostgreSQL

Una vez iniciada la base de datos, vamos a comprobar que FastAPI puede comunicarse correctamente con PostgreSQL.

3.1. Crear el archivo `main.py`

En el directorio raíz del proyecto, creamos el archivo `main.py` con el siguiente código:

```
import os

from fastapi import FastAPI, Depends
from sqlalchemy import create_engine, text
from sqlalchemy.orm import sessionmaker, Session

# Configuración de la conexión a PostgreSQL
DATABASE_URL = (
    f"postgresql+psycopg://autovida_admin:"
    f"{os.environ['POSTGRES_PASSWORD']}"
    f"@localhost:5432/autovida_db"
)

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine,
)

app = FastAPI(title="AUTOVIDA API")


# Dependencia para gestionar las sesiones de la base de datos
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


# Endpoint para comprobar la conexión con PostgreSQL
@app.get("/")
def comprobar_conexion(db: Session = Depends(get_db)):
    db.execute(text("SELECT 1"))

    return {
        "estado": "ok",
        "mensaje": "¡FastAPI conectado con éxito a PostgreSQL!",
    }
```

¿Qué hace este código?

- Crea la conexión entre FastAPI y PostgreSQL mediante SQLAlchemy.
- Configura las sesiones que se utilizarán para interactuar con la base de datos.
- Define el endpoint `/`, que ejecuta una consulta sencilla para comprobar la conexión.
- Devuelve un mensaje de confirmación si la consulta se ejecuta correctamente.

Importante: el ejemplo utiliza el controlador `psycopg` y obtiene la contraseña de una variable de entorno. Antes de ejecutar el servidor, asegúrate de que `POSTGRES_PASSWORD` esté definida en el entorno de Python.

3.2. Iniciar FastAPI

Desde el directorio raíz del proyecto, ejecutamos:

```
uv run fastapi dev main.py
```

Si todo funciona correctamente, FastAPI iniciará el servidor de desarrollo.

Por defecto, estará disponible en:

```
http://127.0.0.1:8000
```

3.3. Comprobar la conexión

Abrimos la siguiente dirección en el navegador:

```
http://127.0.0.1:8000/
```

Si la conexión con PostgreSQL funciona, obtendremos una respuesta similar a esta:

```
{
  "estado": "ok",
  "mensaje": "¡FastAPI conectado con éxito a PostgreSQL!"
}
```

También podemos acceder a la documentación interactiva de la API en:

```
http://127.0.0.1:8000/docs
```

✓ RESUMEN DE COMANDOS

Estos son los comandos principales que utilizaremos durante la configuración inicial y el desarrollo del proyecto.

Acción	Comando
Instalar uv	`curl -LsSf https://astral.sh/uv/install.sh | sh`
Crear el proyecto	`uv init mda-dgp-26/27 –bare`
Acceder al proyecto	`cd mda-dgp-26/27`
Instalar FastAPI	`uv add "fastapi[standard]"`
Instalar dependencias	`uv add sqlalchemy psycopg2-binary alembic pydantic-settings`
Instalar autenticación	`uv add "passlib[bcrypt]"pyjwt`

Acción	Comando
Instalar el controlador moderno	`uv add "psycopg[binary]"`
Iniciar PostgreSQL	`docker compose up -d`
Comprobar los contenedores	`docker ps`
Iniciar FastAPI	`uv run fastapi dev main.py`

► **Resultado esperado**

Al finalizar estos pasos, tendremos:

- ✓ El entorno Python configurado con `uv`.
- ✓ FastAPI instalado y funcionando.
- ✓ PostgreSQL 18 ejecutándose en Docker.
- ✓ Una base de datos llamada `autovida_db`.
- ✓ Una conexión entre FastAPI y PostgreSQL.
- ✓ Un endpoint para comprobar que la conexión funciona.
- ✓ La documentación interactiva de la API disponible en `/docs`.

2.1.3. Guía de uso del backend

Esta guía describe las rutas HTTP que registra actualmente `proyectFastAPI/main.py`, qué usuarios pueden usarlas, qué validan y cómo afectan a los datos. Está escrita a partir del código del backend; no describe pantallas del frontend.

Índice

- Convenciones comunes 14
- Salud de la API 15
- Autenticación 15
- Gestión de usuarios 16
- Asociaciones alumno–tutor/personal 20
- Tareas individuales de alumnos 21
- Clases 25
- Errores y lectura del estado 29
- Interacciones entre módulos 29
- Archivos de implementación 30

Convenciones comunes

- La API se conecta a PostgreSQL mediante una sesión SQLAlchemy por petición. La sesión se cierra al terminar la petición.
- Las rutas protegidas esperan `Authorization: Bearer <token>`. El servidor calcula SHA-256 del token recibido y busca ese hash en `users.token`. El token sin hash solo se devuelve al iniciar sesión.
- Cada usuario tiene un solo token activo. Un login correcto reemplaza el token guardado anteriormente; logout lo pone a `NULL`.
- Los roles reconocidos son `admin`, `staff`, `student` y `parent`.
- Las respuestas correctas usan el sobre `{.errorCode": 0, "details": "successful", ...}`. Los errores usan `{.errorCode": <código>, "details": "<mensaje>"}` y un estado HTTP asociado. Los errores de validación de FastAPI se presentan como código `1005` y HTTP 422. Los errores internos se presentan como código `1500` y HTTP 500, sin exponer detalles SQL.
- Salvo donde se indica otro estado, las rutas correctas responden HTTP 200. Las rutas de creación que lo especifican responden HTTP 201.
- La API ejecuta las mutaciones en transacciones y confirma los cambios con `COMMIT`; cuando detecta un error en una operación transaccional revierte con `ROLLBACK`.

Salud de la API

`GET /` — Comprobar conexión con la base de datos

No requiere autenticación. Ejecuta `SELECT 1` en PostgreSQL y devuelve `estado: "ok"` y `mensaje: "API conectada a PostgreSQL"` dentro del sobre de éxito. Si no se puede ejecutar la consulta, la respuesta pasa por el manejador de error interno.

Entrada: sin cuerpo ni parámetros.

Respuesta JSON:

```
{"errorCode": 0, "details": "successful", "estado": "ok", "mensaje": "API conectada a
↪  PostgreSQL"}
```

Autenticación

`POST /auth/login` — Iniciar sesión

Recibe JSON con `username`, `password` y `role`. Usuario y contraseña deben ser cadenas no vacías; el username no puede contener espacios y no puede superar 50 caracteres. La contraseña no puede superar 72 bytes UTF-8. El rol debe ser uno de los cuatro roles admitidos y no se admiten propiedades JSON adicionales.

El servidor busca la cuenta en `users` junto a la tabla hija correspondiente al rol y compara la contraseña recibida con el hash bcrypt almacenado. Si coincide, genera un token aleatorio, guarda su hash SHA-256 en `users.token` y confirma el cambio. El token anterior de ese usuario deja de servir. La respuesta incluye `access_token`, `token_type: "bearer"` y la identidad `{username, role}`.

Una contraseña incorrecta o una combinación de usuario/rol que no corresponde devuelve `INVALID_CREDENTIALS` (HTTP 401). El endpoint limita `role` a los cuatro valores admitidos; un valor distinto falla la validación de FastAPI y devuelve `INVALID_REQUEST` (HTTP 422). Un fallo de infraestructura de autenticación devuelve `LOGIN_FAILED` (HTTP 500).

JSON de entrada:

```
{"username": "ana", "password": "contraseña", "role": "parent"}
```

Respuesta correcta (el token real es aleatorio):

```
{"errorCode": 0, "details": "successful", "access_token": "<token>", "token_type":
↪  "bearer", "user": {"username": "ana", "role": "parent"}}
```

Ejemplo de error:

```
{"errorCode": 1001, "details": "Usuario o contraseña incorrectos"}
```

`POST /auth/logout` — Cerrar sesión

Requiere un Bearer válido. La dependencia primero verifica que el token corresponde a una sesión activa. Después, el endpoint guarda `NULL` en `users.token` para ese token y confirma la

transacción. La respuesta identifica el usuario y rol cuya sesión se cerró. Si falta el token, no es válido o ya fue revocado, devuelve `INVALID_TOKEN` (HTTP 401).

Entrada: sin cuerpo. Cabecera: `Authorization: Bearer <token>`.

Respuesta JSON:

```
{"errorCode": 0, "details": "successful", "username": "ana", "role": "parent"}
```

Dependencias de autenticación utilizadas por otras rutas

- `get_current_user` valida el Bearer y aporta `username` y `role` a la operación. Un token ausente o inválido impide acceder con HTTP 401.
- `require_admin` además exige el rol `admin`; el resto recibe `ADMIN_REQUIRED` (HTTP 403).
- Un token nuevo invalida el anterior para ese usuario. Cambiar contraseña o username también revoca la sesión al actualizar `users.token`.

Gestión de usuarios

Todas las rutas de este grupo tienen prefijo `/users`.

En este módulo, "avatar" significa la foto de perfil del alumno. Aunque la tabla `Student` la referencia como `avatar_id_fk`, cada alumno debe tener siempre una foto asignada.

`POST /users` — Crear usuario

Solo administrador. Recibe un objeto JSON discriminado por `role`.

Campos comunes obligatorios: `username`, `full_name` y `password`. El username se recorta y no puede contener espacios; el nombre completo se recorta, debe contener alguna letra y no puede contener números. La contraseña no puede superar 72 bytes UTF-8 (límite de bcrypt). No se aceptan campos no definidos.

Para `role: "student"` también son obligatorios todos los campos de accesibilidad: `stud_access_main_col`, `stud_access_sec_col`, `stud_access_ntasks_visible`, `stud_access_pictograms`, `stud_access_contrast`, `stud_access_fontsize`, `stud_access_filters`, `stud_access_ndays_visible`, `stud_access_prof_mod`, `stud_access_timebar`, `stud_access_talkback`, `stud_access_profile_mod` y `stud_access_task_mod`. Los colores deben ser hexadecimales de 3 o 6 dígitos (por ejemplo, `#abc` o `#aabbcc`). Las cantidades visibles deben ser enteros JSON no negativos y el tamaño de fuente un entero JSON positivo. Los campos booleanos requieren `true` o `false` de JSON.

Para `admin`, `staff` y `parent`, también es obligatorio `email`, que debe tener formato válido y no exceder 254 caracteres.

La operación inserta la fila común en `users` (incluido el hash bcrypt de la contraseña y `full_name`) y después la especialización del rol en `Admn`, `Staff`, `Student` o `Parents`, con los datos de email o accesibilidad que correspondan. Al crear un alumno, también crea y asigna una foto de perfil SVG predeterminada en `Avatar`; el alumno nunca se crea con `avatar_id_fk` nulo. Todas las inserciones se confirman conjuntamente. El username es único en la tabla común, por lo que no puede repetirse entre roles. Según el conflicto, se devuelve `USERNAME_TAKEN` o `EMAIL_TAKEN` (HTTP 409); otros errores de creación usan `USER_CREATE_FAILED` (HTTP 500).

El rol se fija al crear la cuenta y es inmutable. Ni el propio usuario ni un administrador pueden cambiarlo después; para cambiar el rol hay que crear una cuenta nueva con el rol correcto.

JSON de entrada para estudiante:

```
{"username":"luis","full_name":"Luis Pérez","password":"clave","role":"student","stud_ac
↪ cess_main_col":"#ffffff","stud_access_sec_col":"#000000","stud_access_ntasks_visible
↪ ":5,"stud_access_pictograms":true,"stud_access_contrast":false,"stud_access_fontsize
↪ ":18,"stud_access_filters":true,"stud_access_ndays_visible":7,"stud_access_prof_mod"
↪ :false,"stud_access_timebar":true,"stud_access_talkback":false,"stud_access_profile_
↪ mod":true,"stud_access_task_mod":true}
```

JSON de entrada para admin, staff o parent:

```
{"username":"ana","full_name":"Ana
↪ López","password":"clave","role":"parent","email":"ana@example.com"}
```

Respuesta correcta:

```
{"errorCode": 0, "details": "successful"}
```

`GET /users` y `GET /users?role=<rol>` — Listar usuarios

Solo administrador. Sin `role`, reúne usuarios de las cuatro tablas hijas; con `role`, filtra a un solo rol. Cada resultado contiene username, full name, rol y email cuando el rol tiene correo. Para estudiantes también incluye sus atributos de accesibilidad. La lista se ordena por username. No devuelve contraseñas ni tokens.

El parámetro `role` se valida contra los cuatro roles admitidos; un valor distinto devuelve `INVALID_REQUEST` (HTTP 422). Los fallos de consulta devuelven `USER_LIST_FAILED` (HTTP 500).

Entrada: sin cuerpo. Filtro opcional en query, por ejemplo `?role=student`.

Respuesta JSON (ejemplo abreviado):

```
{"errorCode":0,"details":"successful","users":[{"username":"ana","full_name":"Ana López"
↪ ,"role":"parent","email":"ana@example.com"},{"username":"luis","full_name":"Luis
↪ Pérez","role":"student","stud_access_main_col":"#ffffff","stud_access_sec_col":"#000
↪ 000","stud_access_ntasks_visible":5,"stud_access_pictograms":true,"stud_access_contr
↪ ast":false,"stud_access_fontsize":18,"stud_access_filters":true,"stud_access_ndays_v
↪ isible":7,"stud_access_prof_mod":false,"stud_access_timebar":true,"stud_access_talkb
↪ ack":false,"stud_access_profile_mod":true,"stud_access_task_mod":true}]}
```

`GET /users/{role}/{username}` — Consultar una cuenta concreta

Solo administrador. Busca una cuenta por username dentro del rol indicado y devuelve sus datos de perfil: `username`, `full_name`, `role`, email cuando corresponda y preferencias de accesibilidad para estudiantes. No devuelve contraseña ni token. Si la cuenta no existe en ese rol, responde `USER_NOT_FOUND` (HTTP 404); si falla la consulta, `USER_LOOKUP_FAILED` (HTTP 500).

Entrada: sin cuerpo. Ejemplo: `GET /users/parent/ana` con Bearer de administrador.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","username":"ana","full_name":"Ana
↪ López","email":"ana@example.com","role":"parent"}
```


`GET /users/me` — Consultar mi propia ficha

Requiere autenticación y devuelve únicamente la ficha del usuario asociado al token. El endpoint obtiene el rol y el username de la sesión; el cliente no puede indicar otra cuenta. Para el alumnado incluye los ajustes de accesibilidad (`stud_access_*`), de modo que pueda cargar su configuración al iniciar sesión. Para los demás roles devuelve los campos propios de su perfil. No devuelve contraseña ni token. Una sesión ausente o inválida recibe `INVALID_TOKEN` (HTTP 401); si no se encuentra la ficha asociada a la sesión, responde `USER_NOT_FOUND` (HTTP 404).

Entrada: sin cuerpo ni parámetros. Ejemplo: `GET /users/me` con el Bearer del alumno autenticado.

Respuesta JSON (ejemplo de alumno):


```
{"errorCode":0,"details":"successful","username":"luis","full_name":"Luis
↪ Pérez","stud_access_main_col":"#ffffff","stud_access_sec_col":"#000000","stud_access
↪ _ntasks_visible":5,"stud_access_pictograms":true,"stud_access_contrast":false,"stud_
↪ access_fontsize":18,"stud_access_filters":true,"stud_access_ndays_visible":7,"stud_a
↪ ccess_prof_mod":false,"stud_access_timebar":true,"stud_access_talkback":false,"stud_
↪ access_profile_mod":true,"stud_access_task_mod":true,"role":"student"}
```


`PUT /users/{role}/{username}` — Editar perfil o datos de usuario

Requiere autenticación. Un administrador puede editar cualquier cuenta. Un `parent` o `staff` solo puede editar su email o contraseña y no puede enviar `username`, `full_name` ni `role`. Un `student` solo puede editar sus propios ajustes de accesibilidad, y únicamente si `stud_access_profile_mod` está activado; si está desactivado, se rechaza la edición con `PROFILE_UPDATE_DENIED` (HTTP 403). El alumno no puede cambiar `username`, `full_name`, contraseña ni `role`, ni modificar los indicadores de permisos `stud_access_prof_mod`, `stud_access_profile_mod` y `stud_access_task_mod`; estos campos quedan reservados al administrador. Los estudiantes no admiten email. Ningún usuario puede cambiar el rol de una cuenta: un administrador que envíe un `role` distinto del actual recibe `ROLE_CHANGE_DENIED` (HTTP 403); un usuario no administrador que envíe `role` recibe `PROFILE_UPDATE_DENIED` (HTTP 403).

El JSON acepta campos opcionales: `username`, `role`, `full_name`, `password`, `email` y campos `stud_access_*`. No se aceptan propiedades extra. Debe haber al menos un campo que modificar. El username y el nombre completo aplican validaciones iguales a las de creación; la contraseña está limitada a 72 bytes. Los valores numéricos y booleanos de accesibilidad deben usar sus tipos JSON correspondientes, y los colores deben seguir el formato hexadecimal indicado en creación.

Cambiar username actualiza la clave común y las referencias en cascada; también actualiza `Tasks.task_creator` y `Alt_Tasks.alt_task_creator`. Cambiar username o contraseña invalida el token activo. Cambiar solo `full_name`, email o accesibilidad no invalida el token. El rol y su fila de especialización no se modifican mediante esta ruta.

Los roles con email deben conservar o recibir un email válido; estudiantes no admiten email. La respuesta de éxito devuelve el `username` y el rol resultante.

Errores relevantes: `PROFILE_UPDATE_DENIED` (403), `ROLE_CHANGE_DENIED` (403), `USER_NOT_FOUND` (404), `USERNAME_TAKEN/EMAIL_TAKEN` (409), `NO_UPDATE_FIELDS`, `INVALID_REQUEST`, `EMAIL_REQUIRED`, `EMAIL_UNAVAILABLE` y `LAST_ADMIN` (400 según catálogo).

JSON de entrada (ejemplo de cambios parciales; solo se envían los campos que se desean modificar):

```
{"full_name":"Nombre actualizado","email":"nuevo@example.com","password":"nueva-clave"}
```

Respuesta correcta:

```
{"errorCode":0,"details":"successful","username":"ana","role":"parent"}
```

`DELETE /users/{role}/{username}` — Eliminar usuario

Solo administrador. Borra la cuenta de `users` únicamente si existe la especialización indicada en la tabla hija del rol. Las claves foráneas determinan la eliminación de los datos dependientes. Para administradores, bloquea la tabla de administradores y comprueba que no sea el último antes de borrar. Si la cuenta no existe devuelve `USER_NOT_FOUND`; si se intenta borrar el último admin, `LAST_ADMIN`. El token de la cuenta borrada deja de poder validarse al desaparecer el usuario.

Entrada: sin cuerpo. Ejemplo: `DELETE /users/parent/ana`.

Respuesta correcta:

```
{"errorCode":0,"details":"successful"}
```

`PUT /users/students/{student_username}/avatar` — Actualizar foto de perfil de alumno

Solo administrador. Recibe `multipart/form-data` con el archivo en el campo `file` y el username del alumno en la ruta. Admite PNG, JPEG y WebP de hasta 5 MB; el backend comprueba la firma y estructura básica del formato, y rechaza formatos no admitidos o archivos demasiado grandes con `AVATAR_INVALID_IMAGE` (HTTP 422) o `AVATAR_TOO_LARGE` (HTTP 413). Si el alumno no existe, devuelve `USER_NOT_FOUND`. El permiso `stud_access_profile_mod` no limita al administrador. El archivo es obligatorio, así que esta operación siempre reemplaza la foto por otra válida y nunca la deja nula.

La imagen se guarda en `Avatar.avatar_bin` y su clave se vincula a `Student.avatar_id_fk` en una transacción. Al reemplazar una foto anterior, el backend la elimina si ya no está vinculada a ningún alumno. La columna `avatar_id_fk` es obligatoria y la clave foránea impide borrar una imagen mientras un alumno la use. La respuesta JSON incluye `avatar_id`, `media_type` y `avatar_url`.

La ruta devuelta en `avatar_url` permite recargar la imagen; evita caché con `Cache-Control: no-store`.

Entrada: `PUT /users/students/luis/avatar`, cabecera `Authorization: Bearer <token>` de administrador y `multipart/form-data`, campo `file` con los bytes de la imagen; no es JSON.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","avatar_id":12,"media_type":"image/png","avatar_ur
↪ l":"/users/students/luis/avatar-image"}
```

`GET /users/students/me/avatar` — Obtener mi avatar

Requiere token de alumno y devuelve directamente la imagen con su tipo MIME. Si el alumno todavía no tiene una foto válida, devuelve una silueta SVG predeterminada para que el perfil siempre pueda mostrar una imagen. La respuesta usa `Cache-Control: no-store` para que una foto recién actualizada no se quede en caché.

Entrada: sin cuerpo. Cabecera: `Authorization: Bearer <token>`.

Respuesta: contenido binario de imagen (PNG, JPEG o WebP), o SVG predeterminado; no devuelve JSON. El encabezado `Content-Type` indica el formato y `Cache-Control: no-store` evita caché.

`GET /users/students/{student_username}/avatar-image` — Obtener avatar de alumno de una clase

Puede consultarse sin token para mostrar la selección de alumno antes del login, siempre que el alumno esté matriculado en alguna clase. El personal solo puede consultar el avatar de alumnos que estén matriculados en alguna de sus clases; el administrador puede consultar el de cualquier alumno. Devuelve la imagen almacenada o el SVG predeterminado si el alumno no tiene avatar. La respuesta usa `Cache-Control: no-store`. Sin autorización para consultar al alumno, devuelve `ASSOCIATION_ACCESS_DENIED` (HTTP 403).

Entrada: sin cuerpo; username en la ruta. La cabecera Bearer es opcional para alumnos matriculados.

Respuesta: contenido binario de imagen (PNG, JPEG o WebP), o SVG predeterminado; no devuelve JSON.

Asociaciones alumno–tutor/personal

Todas las rutas usan el prefijo `/student-links`. La relación alumno-tutor se almacena directamente en `par_stud`. La relación de un alumno con el personal se deriva de `class_stud` y `class_staff`: el alumno y los miembros del personal que imparten su clase quedan relacionados sin una tabla de enlace directa. No se crean ni borran cuentas desde este módulo.

`POST /student-links/students/{student_username}/parents/{parent_username}` — Asociar tutor a alumno

Solo administrador. Comprueba que existen el alumno y el tutor en `users` y sus tablas de rol. Inserta la relación alumno–tutor. Si ya existe, `ON CONFLICT DO NOTHING` evita duplicarla; la respuesta sigue siendo exitosa e indica `association_created: false`. Si se creó, indica `true`.

Entrada: sin cuerpo; usernames en la ruta.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","association_created":true,"student_username":"lui
↪  s","parent_username":"ana","relation":"tutor"}
```


`DELETE /student-links/students/{student_username}/parents/{parent_username}` — Desasociar tutor de alumno

Solo administrador. Elimina únicamente esa relación de `par_stud`. Si no existía, devuelve `ASSOCIATION_NOT_FOUND` (HTTP 404). No borra el alumno, el tutor ni otras relaciones.

Entrada: sin cuerpo; usernames en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","student_username":"luis","parent_username":"ana"}
```

`GET /student-links/students/{student_username}` — Consultar tutores y personal de un alumno

Solo administrador. Devuelve dos listas actuales: `parents` y `staff`. Cada entrada incluye username, full name y email. Los padres salen de `par_stud`; el personal se obtiene de la clase a la que pertenece el alumno y sus filas en `class_staff`. Si el alumno aún no tiene clase, `staff` será una lista vacía. Si no existe el alumno, devuelve `ASSOCIATED_ENTITY_NOT_FOUND` (HTTP 404).

Entrada: sin cuerpo; username en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","student_username":"luis","parents":[{"username":"
↪ ana","full_name":"Ana
↪ López","email":"ana@example.com"}],"staff":[{"username":"marta","full_name":"Marta
↪ Ruiz","email":"marta@example.com"}]}
```

`GET /student-links/parents/{parent_username}/students` — Consultar alumnos de un tutor

Requiere token. Un tutor consulta su propio listado; un administrador puede consultar cualquier tutor. Otros roles, o un tutor que intente consultar otro username, reciben `ASSOCIATION_ACCESS_DENIED` (HTTP 403). Devuelve `username` y `full_name` de cada alumno asociado, ordenados por username. Sin asociaciones, devuelve `students: []`.

Entrada: sin cuerpo; cabecera Bearer y username del tutor en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","parent_username":"ana","students":[{"username":"l
↪ uis","full_name":"Luis Pérez"}]}
```

`GET /student-links/staff/{staff_username}/students` — Consultar alumnos de un miembro del personal

Staff consulta su propio listado; admin puede consultar cualquier miembro del personal. Devuelve los alumnos inscritos en cualquiera de las clases que imparte ese miembro del personal, con `username` y `full_name`, sin duplicados. Un resultado vacío es `students: []`.

Entrada: sin cuerpo; cabecera Bearer y username del miembro del personal en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","staff_username":"marta","students":[{"username":"
↪ luis","full_name":"Luis Pérez"}]}
```

Las relaciones vigentes también controlan las tareas individuales. El tutor debe seguir asociado al alumno mediante `par_stud`; el personal debe compartir con el alumno una clase vigente, comprobada mediante `class_staff` y `class_stud`. Al retirar al alumno de la clase o al personal de ella, desaparece ese acceso derivado. El cambio no afecta a relaciones con padres ni a otras clases.

Tareas individuales de alumnos

Prefijo `/student-tasks`. El personal (`staff`) puede crear tareas para alumnos inscritos en alguna clase que imparte; los tutores (`parent`) pueden hacerlo para alumnos enlazados mediante `par_stud`. El alumno puede crear y eliminar solo sus propias tareas mediante las rutas `/me`, si `stud_access_task_mod` está activado. El administrador no está autorizado por este router. Las rutas de edición de tareas alternativas siguen reservadas a `staff` y `parent`; también validan la relación actual con al menos un alumno asignado a la tarea.

`GET /student-tasks/students/{student_username}?task_date=AAAA-MM-DD` — Consultar tareas de un alumno por día

Requiere personal asignado a una clase compartida con el alumno o tutor asociado mediante `par_stud`. Devuelve las tareas de la agenda de ese alumno para la fecha indicada, ordenadas según `task_index`; incluye también `class_name_fk` para identificar las asignaciones que proceden de una clase. Si el alumno no existe, devuelve `TASK_STUDENT_NOT_FOUND`; si el usuario autenticado no está asociado a él, devuelve `TASK_STUDENT_NOT_ASSOCIATED`.

Entrada: sin cuerpo. Ejemplo: `GET /student-tasks/students/luis?task_date=2026-11-10`.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","stud_usr_fk":"luis","task_date":"2026-11-10","tas
↪ ks":[{"task_id":41,"task_title":"Leer un
↪ cuento","task_start_hour":"10:00:00","task_duration":20,"task_creator":"marta","task
↪ _finished":false,"alt_task_id_fk":null,"task_date":"2026-11-10","task_index":0,"clas
↪ s_name_fk":"Primero A"}]}
```


`GET /student-tasks/me?task_date=AAAA-MM-DD` — Consultar tareas propias por día

Requiere token de alumno. Todos los alumnos pueden consultar su agenda, independientemente del permiso `stud_access_task_mod`, que solo controla la creación o modificación de sus propias tareas. Devuelve las tareas de la agenda propia para esa fecha, en orden de `task_index`. Si no hay tareas para ese día, `tasks` es una lista vacía.

Entrada: sin cuerpo. Ejemplo: `GET /student-tasks/me?task_date=2026-11-10`.

Respuesta JSON: usa el mismo formato de la consulta de tareas de un alumno.

`POST /student-tasks/students/{student_username}` — Crear tarea en la agenda de un alumno

Responde HTTP 201. El JSON requiere `task_title`, `task_date` y `task_duration`. La fecha es obligatoria, puede ser futura y no se limita al día actual. El título se recorta y debe contener entre 1 y 100 caracteres; la duración es un entero positivo expresado en minutos. Son opcionales `task_start_hour` (hora local sin zona) y `alt_task_id_fk` (entero positivo). `task_index` no se envía: el backend lo calcula para la agenda del alumno. Si falta la duración, es `null` o no es positiva, FastAPI rechaza la petición con error de validación `1005` (HTTP 422). No admite propiedades adicionales.

El alumno debe existir y el usuario autenticado debe estar asociado a él según su rol. El personal debe indicar `task_start_hour`; el intervalo completo `[hora de inicio, hora de inicio + duración)` debe quedar entre las 09:00 y las 15:00. `parent` puede indicar una hora, pero su intervalo completo no puede solaparse con `[09:00, 15:00)`. La franja escolar se aplica todos los días; por ahora tampoco se admiten tareas de tutor entre las 09:00 y las 15:00 en fin de semana. Los festivos no se distinguen hasta disponer de un calendario del centro. Las tareas de `parent` sin hora se permiten y se colocan después de todas las tareas con hora de esa fecha. En los límites, terminar exactamente a las 09:00 o empezar exactamente a las 15:00 no se considera solapamiento para

tutores; el personal sí puede terminar exactamente a las 15:00. Si se indica alternativa, esta debe existir y no estar ya enlazada a otra tarea.

La operación inserta en `Tasks` con `task_creator` igual al username del creador, `task_duration` positiva y `task_finished = false`, añade el subtipo `Generic_Task` y crea la asignación `stud_task` con alumno, tarea, índice calculado y fecha. Las tareas con hora se insertan según su hora respecto a las tareas de esa fecha; las tareas sin hora de `parent` se colocan al final, por lo que su índice queda después de la última tarea de `staff`. Las tareas de `parent` con hora anterior a las 09:00 quedan antes de las de `staff`, y las posteriores a las 15:00 quedan después. Si se inserta una tarea antes de otras, sus índices se desplazan para abrirle sitio. La agenda del alumno se bloquea durante el cálculo para serializar inserciones concurrentes. Todo se confirma en una transacción. La respuesta incluye el índice asignado. El esquema también impone `NOT NULL` y `CHECK (> 0)` en `Tasks.task_duration` y `Alt_Tasks.alt_task_duration`; las alternativas ya almacenadas deben cumplirlo aunque esta API solo las asocie.

`CreacionTablas.sqlscript` recrea el esquema completo: elimina primero las tablas existentes y después vuelve a crearlas. Es destructivo y borra todos los datos de esas tablas; se debe ejecutar solo cuando se quiera reiniciar la base de datos.

La hora de inicio es opcional para `parent` y `student`; si se omite, la tarea se coloca al final de la agenda. `staff` debe indicar la hora de inicio y su tarea debe quedar dentro del horario escolar. Tanto `parent` como `student` pueden indicar una hora fuera del horario escolar, validándose el intervalo completo.

JSON de entrada:

```
{"task_title":"Leer un cuento","task_date":"2026-11-10","task_duration":20,"task_start_h
↪ our":"16:30:00","alt_task_id_fk":3}
```

Respuesta JSON (valores de ejemplo):

```
{"errorCode":0,"details":"successful","task_id":41,"task_title":"Leer un cuento","task_s
↪ tart_hour":"16:30:00","task_duration":20,"task_creator":"ana","task_finished":false,
↪ "alt_task_id_fk":3,"stud_usr_fk":"luis","task_index":0,"task_date":"2026-11-10"}
```

`POST /student-tasks/me` — Crear una tarea propia

Requiere token de alumno. El JSON acepta título, fecha, duración positiva en minutos y, opcionalmente, hora de inicio y alternativa. Si se indica hora, el intervalo completo no puede solaparse con `[09:00, 15:00)`; si no se indica, la tarea se trata como tarea solo de duración y se coloca al final de la agenda, después de las tareas de `staff`. Una tarea con hora antes de las 09:00 queda antes de las tareas de `staff`; una tarea posterior a las 15:00 queda después. El alumno debe tener `stud_access_task_mod = true`; si no, responde `STUDENT_TASK_MODIFICATION_DENIED` (HTTP 403). Se crea una tarea `Generic_Task` con `task_creator` igual al username del alumno y una asignación `stud_task` para ese mismo alumno. Devuelve HTTP 201.

JSON de entrada:

```
{"task_title":"Dibujar","task_date":"2026-11-10","task_start_hour":"16:30:00","task_dura
↪ tion":20}
```

Respuesta JSON (valores de ejemplo):

```
{"errorCode":0,"details":"successful","task_id":42,"task_title":"Dibujar","task_start_ho
↪ ur":"16:30:00","task_duration":20,"task_creator":"luis","task_finished":false,"alt_t
↪ ask_id_fk":null,"stud_usr_fk":"luis","task_index":0,"task_date":"2026-11-10"}
```


`PUT /student-tasks/{task_id}/alternative` — Cambiar tarea alternativa

Requiere JSON con `alt_task_id_fk`: un entero positivo para asociar una alternativa o `null` para quitarla. Solo permite modificar una tarea que haya creado el usuario autenticado y que esté asignada al menos a un alumno que siga asociado a él. La alternativa debe existir y no estar asignada a otra tarea. Devuelve los datos actualizados de la tarea. Si la alternativa está ocupada, devuelve `ALTERNATIVE_TASK_ALREADY_ASSIGNED` (HTTP 409); si la tarea principal no es accesible o no existe, devuelve `TASK_NOT_FOUND`.

JSON de entrada:

```
{"alt_task_id_fk":3}
```

Para quitar la alternativa: `{."alt_task_id_fk": null}`.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","task_id":41,"task_title":"Leer un
↪ cuento","task_start_hour":"16:30:00","task_duration":20,"task_creator":"ana","task_f
↪ inished":false,"alt_task_id_fk":3}
```


`DELETE /student-tasks/students/{student_username}/{task_id}?task_date=AAAA-MM-DD` — Quitar tarea de la agenda del alumno

El `task_date` de la query identifica la fecha de la asignación. El índice ya no se envía, porque puede cambiar al insertar tareas anteriores en la agenda. El backend localiza el índice vigente por alumno, tarea y fecha. Comprueba que el alumno exista, que siga asociado al usuario y que exista una fila `stud_task` correspondiente.

Si se conoce la hora de inicio, un tutor no puede quitar una tarea dentro del horario escolar y el personal no puede quitarla fuera del horario escolar (09:00 inclusive, 15:00 exclusive). Si la hora es `NULL`, se omite esa regla. La operación elimina únicamente la fila `stud_task`, por lo que deja de estar asignada a ese alumno, pero no borra la tarea de `Tasks` ni el registro genérico. Devuelve los identificadores de la asignación quitada.

Entrada: sin cuerpo. Ejemplo de ruta y query: `DELETE /student-tasks/students/luis/41?task_date=2026-11-10`.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","task_id":41,"stud_usr_fk":"luis","task_date":"202
↪ 6-11-10","task_index":0}
```


`DELETE /student-tasks/me/{task_id}?task_date=AAAA-MM-DD` — Eliminar una tarea propia

Requiere token de alumno y `stud_access_task_mod = true`. Comprueba que la asignación corresponda al alumno autenticado y que `Tasks.task_creator` también sea su username, para impedir que borre tareas de personal o tutores. Si la tarea tiene hora, su intervalo completo debe

quedar fuera de [09:00, 15:00); una tarea solo de duración sin hora también se puede eliminar. Si el intervalo se solapa con el horario escolar, responde STUDENT_TASK_OUTSIDE_SCHOOL_HOURS (HTTP 400). Sin permiso de modificación responde STUDENT_TASK_MODIFICATION_DENIED (HTTP 403). Elimina la asignación indicada y, si la tarea propia ya no tiene ninguna otra asignación, elimina también su fila de Tasks y el Generic_Task asociado. Devuelve los identificadores de la asignación eliminada.

Entrada: sin cuerpo. Ejemplo: DELETE /student-tasks/me/42?task_date=2026-11-10.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","task_id":42,"stud_usr_fk":"luis","task_date":"202
↪ 6-11-10","task_index":0}
```

Clases

Prefijo /classes. Las clases se guardan en Class; class_staff asocia personal a clases, class_stud inscribe alumnos y class_task vincula tareas grupales a una clase. stud_task.class_name_fk marca las asignaciones a alumnos que proceden de una clase; las tareas individuales dejan ese campo nulo. El personal solo puede consultar o administrar una clase si está asignado a ella. El administrador puede consultar todas las clases y sus detalles, gestionar el catálogo y las asignaciones de personal, e inscribir o retirar alumnos de cualquier clase.

POST /classes — Crear clase

Solo administrador. JSON { çlass_name": "..."}. El nombre se recorta, debe quedar no vacío y no superar 50 caracteres. Crea la clase; si ya existe, devuelve CLASS_NAME_TAKEN (HTTP 409). Responde HTTP 201.

JSON de entrada:

```
{"class_name":"Primero A"}
```

Respuesta JSON:

```
{"errorCode":0,"details":"successful","class_name":"Primero A"}
```

GET /classes — Listar clases

La consulta es pública para permitir la selección de clase antes del login. Sin token, devuelve únicamente el campo class_name para cada clase, sin otros datos. Con token, el administrador recibe todas las clases y el personal únicamente las clases que tiene asignadas; otros roles reciben CLASS_ACCESS_DENIED (HTTP 403). Cada clase autenticada incluye class_name, student_count y students; cada alumno incluye username, full_name y avatar_url. El endpoint de avatar permite acceso anónimo solo a alumnos matriculados en alguna clase. Si el usuario autenticado no tiene clases visibles, classes es una lista vacía; si una clase no tiene alumnos, students es una lista vacía y student_count vale 0.

Entrada: sin cuerpo; cabecera Bearer opcional.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","classes":[{"class_name":"Primero
↪  A"},{"class_name":"Segundo B"}]}
```

`GET /classes/{class_name}` — Consultar detalle de clase

Puede consultarse sin token para mostrar los alumnos de la clase elegida antes del login. El administrador y las solicitudes anónimas pueden consultar cualquier clase; el personal autenticado solo puede consultar una clase a la que esté asignado. Devuelve el nombre y los alumnos inscritos (`username`, `full_name`, `avatar_url`); no devuelve el personal asignado. Una clase inexistente da `CLASS_NOT_FOUND`; una clase existente a la que ese staff no está asignado da `CLASS_ACCESS_DENIED`. `students` puede estar vacía.

Entrada: sin cuerpo; cabecera Bearer opcional y nombre de clase en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","class_name":"Primero
↪  A","students":[{"username":"luis","full_name":"Luis
↪  Pérez","avatar_url":"/users/students/luis/avatar-image"}]}
```

`PUT /classes/{current_class_name}` — Renombrar clase

Solo administrador. Recibe el nuevo nombre en `{ çlass_name": "..."}`. Renombra la clase; las relaciones conservan el vínculo mediante la clave foránea. Si la clase original no existe, devuelve `CLASS_NOT_FOUND`; si el nombre nuevo ya existe, `CLASS_NAME_TAKEN`.

JSON de entrada:

```
{"class_name":"Primero B"}
```

Respuesta JSON:

```
{"errorCode":0,"details":"successful","class_name":"Primero B"}
```

`DELETE /classes/{class_name}` — Eliminar clase

Solo administrador. Elimina la clase, sus matrículas y asignaciones de tareas grupales. Las tareas vinculadas únicamente a esa clase se eliminan; si una tarea también conserva asignaciones directas a alumnos, esas asignaciones y la tarea se mantienen. No elimina las cuentas de alumnos ni de personal. Si no existe, devuelve `CLASS_NOT_FOUND`.

Entrada: sin cuerpo; nombre de clase en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","class_name":"Primero A"}
```

`POST /classes/{class_name}/staff/{staff_username}` — Asignar personal a clase

Solo administrador. Comprueba la clase y que la cuenta exista como miembro de `Staff`, y la añade a `class_staff`. La inserción repetida no crea duplicado y responde correctamente. Errores: `CLASS_NOT_FOUND` o `CLASS_STAFF_NOT_FOUND`.

Entrada: sin cuerpo; clase y username en la ruta.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","class_name":"Primero A","staff_username":"marta"}
```


`DELETE /classes/{class_name}/staff/{staff_username}` — Retirar personal de clase

Solo administrador. Elimina la asignación correspondiente en `class_staff`; no elimina la cuenta del personal, la clase, sus alumnos ni sus tareas. Al retirar la asignación, esa persona deja de acceder a la clase y a los alumnos que se le asociaban por compartirla. Si la clase no existe, devuelve `CLASS_NOT_FOUND`; si el personal no está asignado a ella, devuelve `CLASS_STAFF_NOT_FOUND` (HTTP 404).

Entrada: sin cuerpo; clase y username en la ruta.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","class_name":"Primero A","staff_username":"marta"}
```


`POST /classes/{class_name}/students/{student_username}` — Inscribir alumno en clase

Requiere token de administrador o de personal. El administrador puede inscribir en cualquier clase; el personal solo en una clase a la que esté asignado. Inscribe al alumno en `class_stud`. Comprueba que sea un alumno existente y que no esté inscrito en otra clase. Al matricularlo, también le asigna las tareas que ya estén vinculadas a esa clase en `class_task`, calculando el índice de cada una en su agenda. Repetir la asignación a la misma clase no duplica la matrícula ni las tareas. Responde HTTP 201. Otros roles reciben `CLASS_ACCESS_DENIED` (HTTP 403). Si ya está en otra clase, `STUDENT_ALREADY_IN_CLASS` (HTTP 409); si no existe como alumno, `TASK_STUDENT_NOT_FOUND`.

Entrada: sin cuerpo; clase y username en la ruta.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","class_name":"Primero
↪ A","student_username":"luis","class_tasks_assigned":2}
```


`DELETE /classes/{class_name}/students/{student_username}` — Retirar alumno de clase

Requiere token de administrador o de personal. El administrador puede retirar alumnos de cualquier clase; el personal solo de una clase a la que esté asignado. En una transacción, elimina las asignaciones `stud_task` marcadas como procedentes de esa clase y luego la inscripción `class_stud`. Las tareas individuales del alumno, incluidas las creadas por el mismo profesor, se conservan. La tarea grupal sigue vinculada a la clase y seguirá disponible para los demás alumnos matriculados. Otros roles reciben `CLASS_ACCESS_DENIED` (HTTP 403). Si la relación no existe, devuelve `ASSOCIATION_NOT_FOUND`.

Entrada: sin cuerpo; clase y username en la ruta.

Respuesta JSON:


```
{"errorCode":0,"details":"successful","class_name":"Primero
↪ A","student_username":"luis","class_task_assignments_removed":2}
```

`POST /classes/{class_name}/tasks` — Crear tarea para toda la clase

Requiere staff asignado a la clase. Recibe título, fecha, hora de inicio y duración positiva en minutos; la alternativa es opcional. La fecha es obligatoria y puede ser futura. Si falta la duración o la hora, o la tarea no queda íntegramente entre las 09:00 y las 15:00, se rechaza; también valida el título y que la alternativa exista y no esté ya asignada. `task_index` no se envía: se calcula por separado para cada alumno según sus tareas de esa fecha. La respuesta incluye `task_indexes`, un mapa de username a índice asignado.

La clase debe tener al menos un alumno inscrito; de lo contrario devuelve `CLASS_HAS_NO_STUDENTS`. Crea una tarea con `task_creator` igual al username del miembro del personal, un registro `Generic_Task`, una fila `class_task` que la vincula a la clase y una fila `stud_task` por cada alumno inscrito. Cada fila `stud_task` guarda también el origen en `class_name_fk`. Calcula un índice individual para cada agenda y fecha. La operación es transaccional y responde HTTP 201 con la lista de alumnos incluidos y sus índices. La tarea existe como tarea de clase aunque más adelante se retire a todos los alumnos.

JSON de entrada:

```
{"task_title":"Lectura","task_date":"2026-11-10","task_duration":20,"task_start_hour":"1
↪ 0:00:00","alt_task_id_fk":3}
```

Respuesta JSON (alumnos y valores de ejemplo):

```
{"errorCode":0,"details":"successful","task_id":41,"task_title":"Lectura","task_start_ho
↪ ur":"10:00:00","task_duration":20,"task_creator":"marta","task_finished":false,"alt_
↪ task_id_fk":3,"class_name":"Primero
↪ A","students":["luis","sofia"],"task_indexes":{"luis":2,"sofia":0},"task_date":"2026
↪ -11-10"}
```

`GET /classes/{class_name}/tasks` — Consultar tareas de una clase

Requiere staff autenticado y asignado a la clase. Devuelve las tareas enlazadas mediante `class_task`, su fecha y los alumnos que aún tienen una asignación de esa tarea originada en la clase. No incluye tareas individuales creadas por el personal para alumnos concretos.

Entrada: sin cuerpo; nombre de clase en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","class_name":"Primero
↪ A","tasks":[{"task_id":41,"task_title":"Lectura","task_start_hour":"10:00:00","task_
↪ duration":20,"task_creator":"marta","task_finished":false,"alt_task_id_fk":3,"task_d
↪ ate":"2026-11-10","students":["luis","sofia"]}]}
```

`PUT /classes/{class_name}/tasks/{task_id}` — Editar tarea de clase

Requiere staff asignado a la clase y que la tarea esté vinculada a ella. Recibe el mismo JSON que la creación de tarea grupal. Actualiza los datos comunes de `Tasks` y la fecha de `class_task`, retira las asignaciones originadas por esa clase y las vuelve a crear para los alumnos actualmente matriculados, con índices calculados para sus agendas. Si la misma tarea conserva asignaciones individuales, estas no se borran, aunque sus datos comunes sí reflejan la edición. Usa

los errores TASK_NOT_FOUND, ALTERNATIVE_TASK_NOT_FOUND, ALTERNATIVE_TASK_ALREADY_ASSIGNED y STAFF_TASK_OUTSIDE_SCHOOL_HOURS cuando corresponda.

JSON de entrada:

```
{"task_title":"Lectura comprensiva","task_date":"2026-11-11","task_duration":25,"task_st
↪ art_hour":"10:00:00","alt_task_id_fk":3}
```

Respuesta JSON:

```
{"errorCode":0,"details":"successful","task_id":41,"task_title":"Lectura
↪ comprensiva","task_start_hour":"10:00:00","task_duration":25,"task_creator":"marta",
↪ "task_finished":false,"alt_task_id_fk":3,"class_name":"Primero
↪ A","task_date":"2026-11-11","task_indexes":{"luis":2,"sofia":0}}
```

DELETE /classes/{class_name}/tasks/{task_id} — Desasignar tarea de una clase

Requiere staff asignado a la clase. Elimina la relación class_task y las asignaciones stud_task cuyo origen sea esa clase. Si la tarea no tiene ya asignaciones directas ni otros vínculos de clase, también elimina la tarea común y su subtipo. Una asignación directa del mismo task_id a un alumno se conserva. Si la relación clase-tarea no existe, devuelve TASK_NOT_FOUND.

Entrada: sin cuerpo; clase y task_id en la ruta.

Respuesta JSON:

```
{"errorCode":0,"details":"successful","class_name":"Primero
↪ A","task_id":41,"student_assignments_removed":2}
```

Errores y lectura del estado

Los códigos y textos públicos se mantienen en proyectFastAPI/errors.py. En general, un código 4xx representa credenciales/permisos/entidades/datos no válidos; un 5xx representa fallo interno. No se deben interpretar errores de PostgreSQL directamente desde el frontend: la API traduce errores conocidos al catálogo y oculta los detalles internos.

Las listas y consultas de asociaciones, clases y tareas se generan desde las relaciones actuales de la base de datos. Las rutas de asociación y de clase no incluyen confirmaciones de interfaz; cualquier diálogo de confirmación antes de una operación destructiva debe implementarse en el cliente antes de enviar la petición.

Interacciones entre módulos

1. **Identidad y especialización:** users contiene los campos comunes (username, full_name, contraseña y token); Admn, Staff, Student y Parents determinan el rol y mantienen campos propios. Login y autorización unen la cuenta común con la especialización del rol.

2. **Sesiones y cambios de usuario:** las rutas protegidas consultan el token guardado en users. Editar contraseña o username revoca ese token. Los roles son inmutables después de crear la cuenta; el login siguiente genera un token nuevo.

3. **Permisos y asociaciones:** `par_stud` es la fuente de la relación directa tutor-alumno. El acceso del personal a los alumnos se deriva por la cadena `class_staff` → `class_stud`. Las consultas de asociaciones y las autorizaciones de tareas usan esas relaciones vigentes.

4. **Agenda:** `Tasks` contiene los datos comunes, `Generic_Task` su especialización y `class_task` vincula la tarea a una clase y fecha. `stud_task` enlaza tareas con alumnos, fecha e índice; `class_name_fk` identifica las filas que proceden de una clase y queda nulo para asignaciones individuales. Retirar a un alumno de una clase elimina solo sus filas de agenda originadas en esa clase. Si se borra una tarea grupal, se conservan sus filas de asignación individual; las tareas sin ningún vínculo se eliminan junto a su subtipo.

5. **Clases y asociaciones alumno-persona:** cada alumno puede estar en una clase como máximo; un profesor puede impartir varias clases y cada clase puede tener varios profesores. Por eso todo el personal de una clase comparte acceso derivado a sus alumnos. La relación tutor-alumno permanece separada en `par_stud`. Cambiar las asignaciones de clase modifica el acceso derivado del personal, pero no las cuentas ni las relaciones con padres.

6. **Tareas alternativas:** `alt_task_id_fk` referencia `Alt_Tasks`; el backend impide usar la misma alternativa en más de una tarea. Cambiar username de un usuario actualiza la autoría guardada en `Tasks` y `Alt_Tasks`.

Archivos de implementación

- Registro de rutas y manejadores globales: `src/backend/proyectFastAPI/main.py`.
- Sesiones: `auth/auth_backend.py`, `auth/auth_db.py`.
- Usuarios: `gestion_usuarios/users_mng_backend.py`, `gestion_usuarios/users_mng_db.py`.
- Asociaciones de alumnos: `asociaciones_alumnos/studs_links_backend.py`, `asociaciones_alumnos/studs_links_db.py`.
- Tareas individuales: `tareas alumnos/studs_tasks_backend.py`, `tareas alumnos/studs_tasks_db.py`.
- Clases y tareas de clase: `clases/clases_backend.py`, `clases/clases_db.py`.
- Errores públicos: `errors.py`.

2.1.4. Guía de errores del backend

La API centraliza los códigos y mensajes públicos en `src/backend/proyectFastAPI/errors.py`. El frontend debe basar su lógica en `errorCode`; `details` es un texto legible para mostrar al usuario o registrar. Las respuestas de error tienen esta forma:

```
{
  "errorCode": 1001,
  "details": "Usuario o contraseña incorrectos"
}
```

Las respuestas correctas usan `errorCode: 0` y `details: "successful"`. Los códigos de error son propios de la API y no equivalen al estado HTTP; el estado HTTP se indica aparte en la respuesta. Por ejemplo, `errorCode: 1002` se devuelve con HTTP 401.

Índice

- Errores funcionales 31
- Errores internos 33
- Comportamiento para el frontend 34

Errores funcionales

errorCode	details	HTTP	Significado y uso
1001	Usuario o contraseña incorrectos	401	El nombre de usuario, la contraseña o el rol no identifican una cuenta válida durante el inicio de sesión.
1002	Token ausente o inválido	401	Falta el token Bearer o no coincide con una sesión activa.
1003	Se requiere una sesión de administrador	403	La operación requiere permisos de administrador.
1004	Solo puedes consultar los alumnos asociados a tu propia cuenta	403	Un tutor o miembro del staff intentó consultar asociaciones de otra cuenta.
1005	La solicitud contiene datos inválidos	422	El cuerpo, los parámetros o la ruta no cumplen la validación del endpoint. El detalle no incluye los valores enviados.
1006	Usuario no encontrado	404	No existe la cuenta solicitada por nombre de usuario, o la cuenta que se iba a editar o eliminar.
1007	Los estudiantes no tienen correo electrónico	400	Se intentó consultar o modificar el correo de una cuenta estudiante.
1008	Usuario no encontrado por correo	404	No hay una cuenta del rol indicado con ese correo.
1009	Rol no válido	400	El rol no es uno de `admin`, `staff`, `student` o `parent`.
1010	El correo electrónico es obligatorio	400	Se intentó crear una cuenta de un rol que requiere correo sin proporcionar uno.
1011	No se puede eliminar el último administrador	400	La operación dejaría el sistema sin administradores.

errorCode	details	HTTP	Significado y uso
1012	Debe indicarse al menos un campo para editar	400	La petición de edición no contiene ningún campo modificable.
1013	No se encontró el usuario o la asociación	404	No se encontró el alumno, tutor o miembro del staff necesario para gestionar la asociación.
1014	La asociación no existe	404	Se intentó desenlazar una asociación que no está registrada.
1015	Tipo de asociación directa no válido; solo se pueden asociar tutores (parent)	400	Las relaciones con tutores se gestionan directamente; las relaciones con staff se derivan de las clases.
1016	Ruta no encontrada	404	La URL no corresponde a ningún endpoint disponible.
1017	Se requiere una sesión autorizada de personal, tutor o alumno	403	La ruta no admite el rol autenticado; los alumnos solo tienen acceso a las rutas para sus propias tareas.
1018	Alumno no encontrado	404	No existe el alumno indicado en la operación de tareas.
1019	El alumno no está asociado a tu cuenta o a una de tus clases	403	El tutor no está vinculado directamente al alumno o el miembro del staff no comparte una clase con él.
1020	La tarea alternativa no existe	404	No se encontró la tarea alternativa solicitada.
1021	Tarea no encontrada	404	No se encontró la tarea solicitada.
1022	La tarea alternativa ya está asociada a otra tarea	409	La tarea alternativa ya tiene una asociación incompatible.
1023	Las tareas del personal requieren hora de inicio y deben quedar entre las 09:00 y las 15:00	400	Una tarea de staff no incluye hora de inicio, empieza antes de las 09:00 o termina después de las 15:00.
1024	La tarea completa del tutor no puede coincidir con el horario escolar	400	El intervalo de la tarea del tutor se solapa con el horario escolar.
1025	Los tutores no pueden eliminar tareas programadas durante el horario escolar	400	Un tutor intentó eliminar una tarea dentro del horario escolar.
1026	El personal no puede eliminar tareas programadas fuera del horario escolar	400	Un miembro del staff intentó eliminar una tarea fuera del horario permitido.
1027	Ya existe un usuario con ese nombre de usuario	409	El nombre de usuario ya está ocupado en `users`, por lo que no puede repetirse en ninguna especialización de rol.
1028	Ya existe un usuario con ese correo electrónico en el rol indicado	409	El correo ya está ocupado por otra cuenta del rol de destino.
1029	No tienes permiso para modificar este perfil	403	La cuenta intentó editar un perfil ajeno o un estudiante cuyo `stud_access_profile_mod` está desactivado intentó modificar sus datos.
1030	Clase no encontrada	404	No existe la clase solicitada.

errorCode	details	HTTP	Significado y uso
1031	Ya existe una clase con ese nombre	409	Se intentó crear o renombrar una clase con un nombre ya registrado.
1032	No tienes permiso para gestionar esta clase	403	La cuenta no es personal asignado a la clase, o el rol no está autorizado para la operación.
1033	El alumno ya pertenece a otra clase	409	Se intentó inscribir a un alumno que ya está inscrito en otra clase.
1034	No se encontró el miembro del personal	404	El usuario indicado no tiene perfil de personal.
1035	No se pueden asignar tareas a una clase sin alumnos	400	Se intentó asignar una tarea grupal sin alumnos inscritos.
1036	El archivo debe ser una imagen PNG, JPEG o WebP válida	422	El alumno intentó actualizar su foto con un archivo que no se reconoce como imagen compatible.
1037	La imagen no puede superar los 5 MB	413	El archivo de foto supera el tamaño máximo permitido.
1038	No se permite cambiar el rol de un usuario	403	Un administrador intentó cambiar el rol de una cuenta. Los usuarios no administradores reciben `PROFILE_UPDATE_DENIED` si intentan enviar un rol.
1039	El alumno no tiene permiso para modificar sus tareas	403	`stud_access_task_mod` está desactivado para el alumno que intenta crear o eliminar una tarea propia.
1040	La tarea del alumno con hora de inicio no puede solaparse con el horario escolar	400	El intervalo de una tarea de estudiante con hora se solapa con el horario `[09:00, 15:00)`. Las tareas sin hora se permiten y se colocan al final de la agenda.

Errores internos

Estos códigos indican un fallo del servidor. El frontend puede mostrar un mensaje genérico y permitir reintentar; no debe esperar detalles de SQL ni de la excepción original.

errorCode	details	HTTP	Significado
1500	Error interno del servidor	500	Fallo inesperado no clasificado.
1501	No se pudo consultar el usuario	500	Falló una consulta de usuario por nombre o correo.
1502	No se pudo consultar la lista de usuarios	500	Falló la consulta que lista cuentas.
1503	No se pudo actualizar la contraseña	500	Falló la actualización de contraseña.
1504	No se pudo crear el usuario	500	Falló la creación de la cuenta.
1505	No se pudo eliminar el usuario	500	Falló la eliminación de la cuenta.
1506	No se pudo editar el usuario	500	Falló la edición de la cuenta.
1507	No se pudo actualizar el avatar	500	Falló la actualización del avatar.

errorCode	details	HTTP	Significado
1508	No se pudo iniciar sesión	500	Falló el proceso de autenticación por un problema interno.

Comportamiento para el frontend

- Comprobar `errorCode === 0` para reconocer una respuesta correcta.
- Para un error conocido, usar `errorCode` para decidir el flujo y `details` como texto visible si resulta apropiado.
- Tratar los códigos 15xx como fallos internos; `details` no revela el error de la base de datos.
- Puede haber errores HTTP producidos antes de entrar a un endpoint. También se devuelven con el mismo formato JSON `errorCode` y `details`.

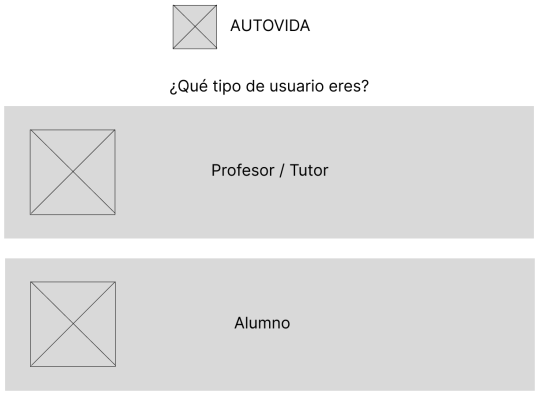
2.2. Documentos generados en el frontend

2.2.1. Flujo de pantallas del frontend

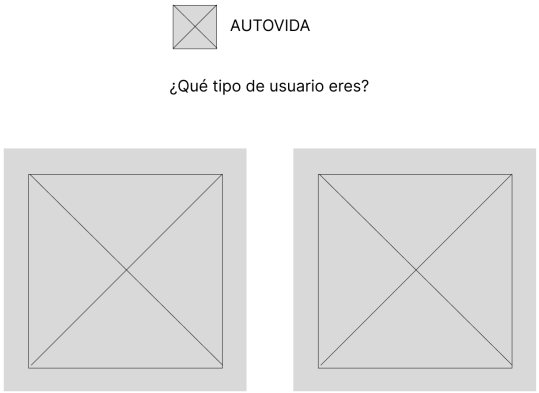
El flujo de pantallas del frontend es demasiado grande para visualizarlo correctamente en este pdf. Se puede encontrar en el repositorio de GitHub, en el archivo docs/Pantallas/flujo-pantallas.png. Se recomienda descargar el archivo para visualizarlo a resolución completa.

2.2.2. Pantallas del frontend

Portada

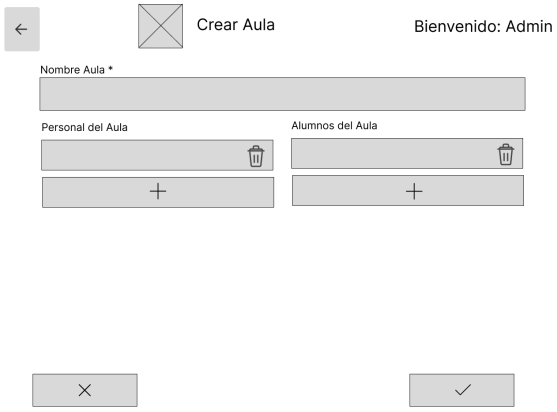


Portada AUTOVIDA V1

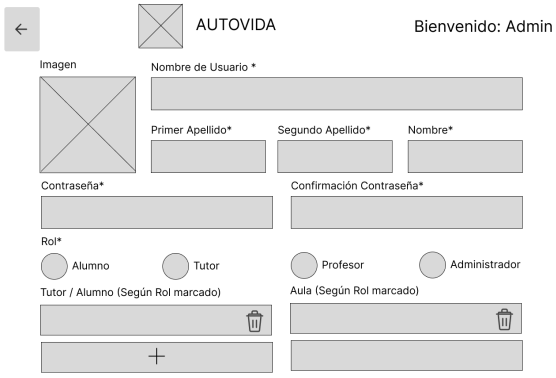


Portada AUTOVIDA V2

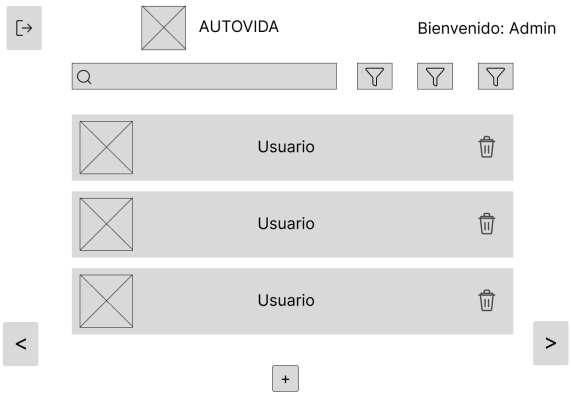
Administración



Crear o modificar aulas

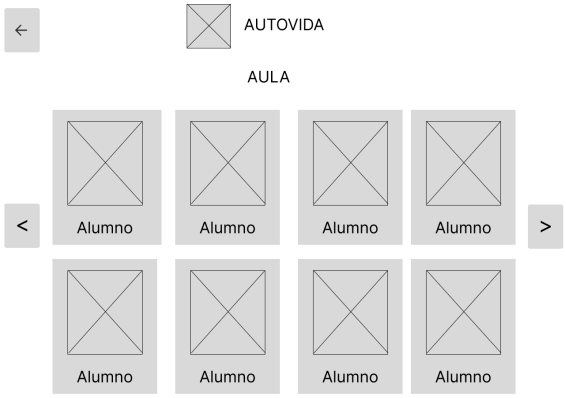


Crear o modificar usuario

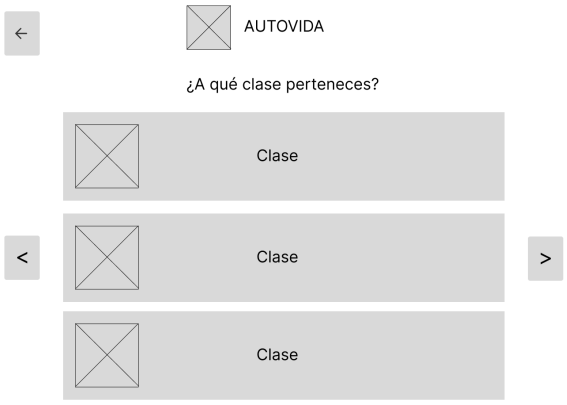


Gestión de usuarios

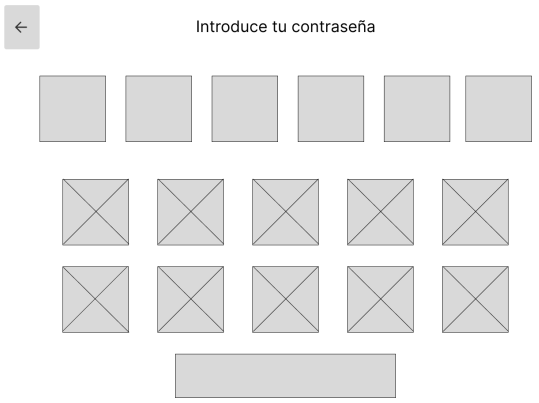
Alumno



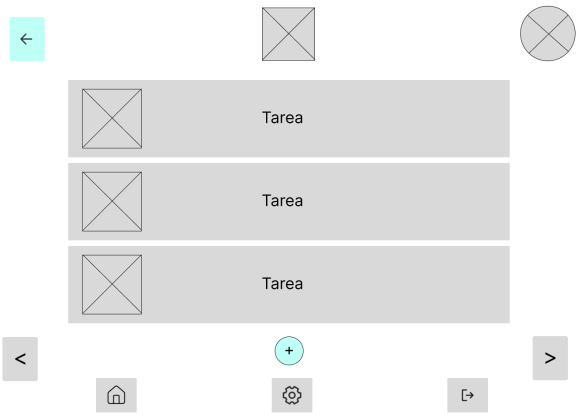
Búsqueda de alumno



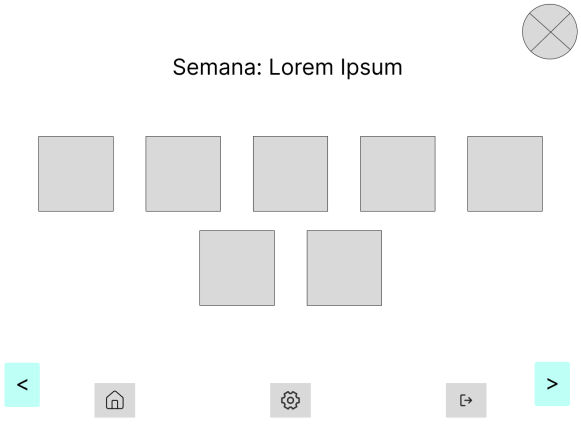
Elección de aulas



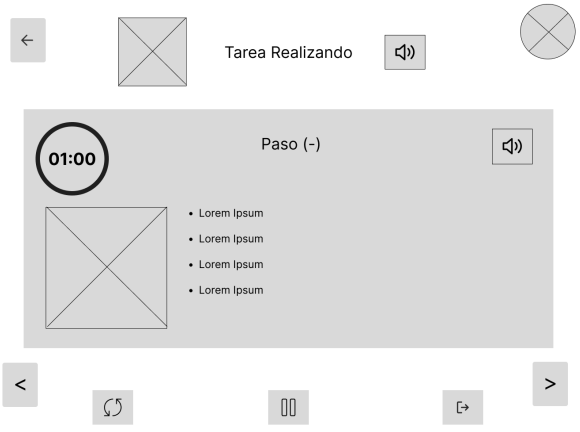
Inicio de sesión



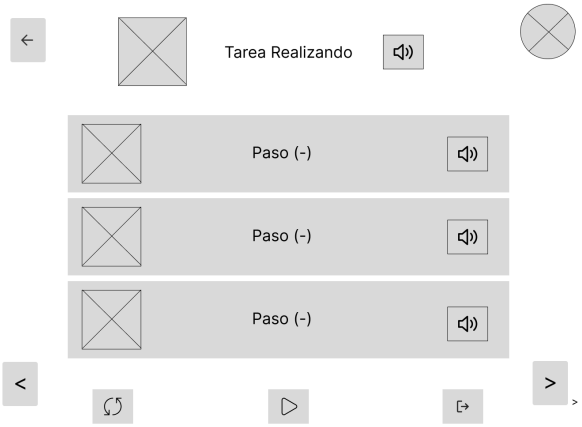
Pantalla inicial



Selección de días

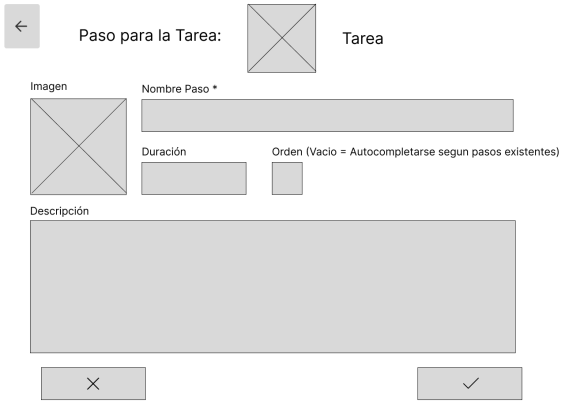


Pantalla de paso

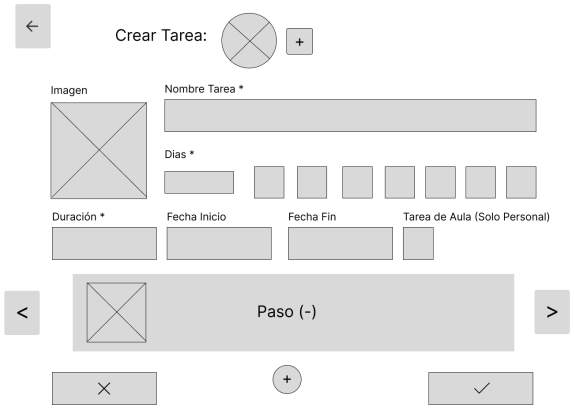


Vista de tarea

Personal y tutor



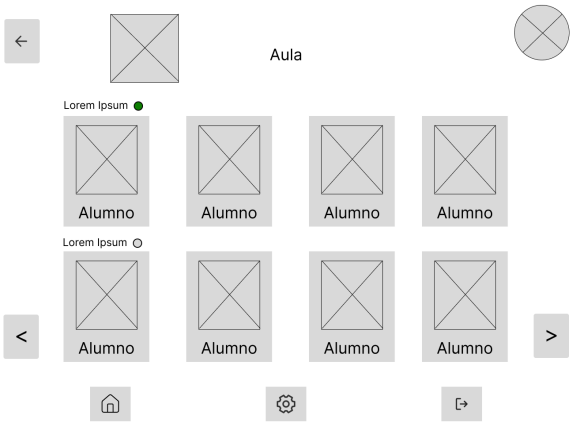
Crear paso



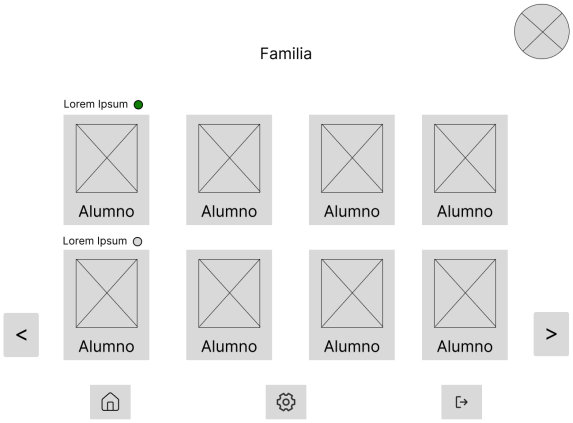
Crear tarea



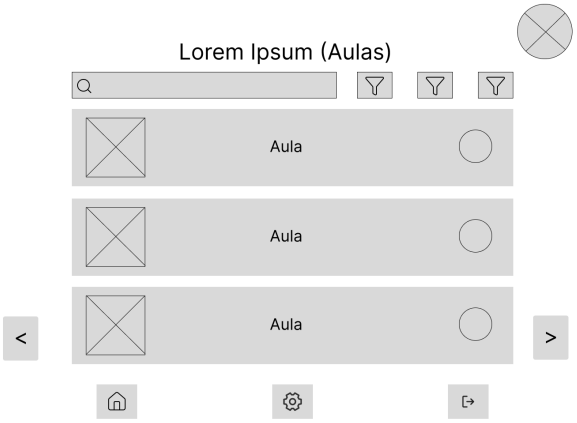
Inicio de sesión



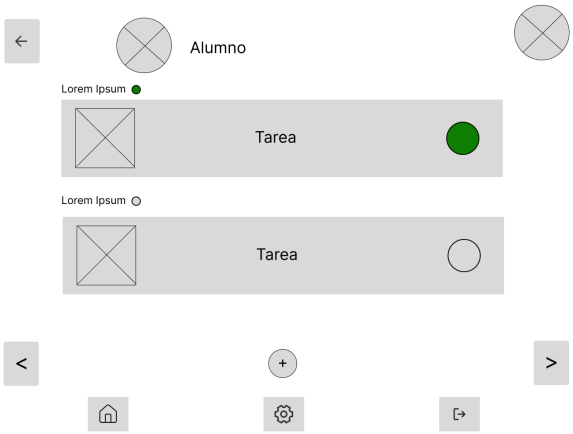
Alumnos del aula



Alumnos del tutor



Aulas del personal



Tareas del alumno

2.2.3. Guía del frontend

AUTOVIDA: guía técnica del frontend

Rama: `iteracion1` · **Revisión:** `11/11/2026` · **Referencia de código:** `0aaf9f1`

Documento de incorporación y mantenimiento de la aplicación Flutter. Describe la organización del proyecto, el funcionamiento de sus componentes principales, el flujo de información entre pantallas y las pruebas de desarrollo. La integración con el backend se limita a las peticiones HTTP utilizadas por el frontend.

La preparación del entorno, el uso del lanzador y la resolución de errores están en la guía de ejecución.

Las rutas de archivos indicadas como código parten de la raíz del repositorio. Los comandos especifican cuándo deben ejecutarse dentro de `src/frontend`.

Índice

- 1. Alcance funcional . . . 39
- 2. Jerarquía del proyecto Flutter . . . 40
- 3. Arquitectura y ciclo de ejecución . . . 43
- 4. Modelos y transformación de datos . . . 45
- 5. Interfaz y componentes compartidos . . . 47
- 6. Integración HTTP utilizada por el frontend . . . 50
- 7. Navegación y flujo entre pantallas . . . 53
- 8. Actualización de datos y persistencia . . . 57
- 9. Pruebas y mantenimiento . . . 59

1. Alcance funcional

AUTOVIDA es una agenda accesible con cuatro perfiles. El rol de la sesión determina la pantalla inicial y las operaciones disponibles.

Perfil	Valor en el código	Pantalla inicial	Funciones
Alumno	`UserRole.student`	`AgendaScreen`	Consultar su agenda y gestionar tareas propias si tiene permiso.
Personal	`UserRole.staff`	`StaffClassesScreen`	Consultar aulas y alumnos asociados, gestionar tareas y retirar alumnos ya matriculados.
Familia	`UserRole.parent`	`FamilyStudentsScreen`	Consultar alumnos vinculados y gestionar sus tareas.
Administración	`UserRole.admin`	`AdminScreen`	Gestionar cuentas, preferencias, fotos, aulas y asociaciones.

El alta de alumnos en un aula está reservada a administración en el frontend. Personal no dispone de controles de alta; la capa de datos del cliente también bloquea esa operación.

La aplicación admite datos reales mediante `LiveRepository` y datos de demostración mediante `DemoRepository`. Las pantallas comparten el mismo contrato de repositorio y no realizan peticiones HTTP directamente.

El recorrido de tareas de esta iteración trabaja con una tarea como unidad. Los modelos de pasos y el temporizador se conservan como utilidades de compatibilidad, pero no hay una pantalla activa de ejecución por pasos.

2. Jerarquía del proyecto Flutter

2.1. Organización general

```
src/frontend/
|-- pubspec.yaml                 Identidad, SDK y dependencias declaradas
|-- pubspec.lock                 Versiones resueltas de las dependencias
|-- analysis_options.yaml        Configuración del analizador y linter
|-- .metadata                    Metadatos de plataformas de Flutter
|-- .gitignore                   Exclusión de artefactos locales
|-- lib/                         Código Dart de la aplicación
|-- test/                        Pruebas unitarias, HTTP y de widgets
|-- linux/                       Integración nativa GTK/CMake
|-- android/                     Integración Android/Kotlin/Gradle
|-- web/                         Contenedor web e iconos
|-- README.md                    Entrada a la documentación
`-- NOTAS_INTEGRACION_MAIN.md     Decisiones y límites de integración
```

Esta guía se encuentra en `docs/explicacionfrontend/GUIA_FRONTEND.md`, fuera del proyecto Flutter.

`lib/` contiene la lógica compartida entre plataformas. Las carpetas `linux/`, `android/` y `web/` aportan el contenedor nativo, permisos y recursos de cada destino; no duplican las pantallas Dart.

`build/`, `.dart_tool/`, los directorios `ephemeral` y los archivos `generated_*` se regeneran durante la resolución de dependencias o la compilación. No son el origen de la interfaz. Este checkout no incluye contenedores iOS, macOS o Windows.

2.2. Árbol del código Dart

```
lib/
|-- main.dart
|-- app.dart
|-- models/
|   |-- domain.dart
|   |-- student_search.dart
|   |-- task_timer.dart
|   `-- validation.dart
|-- services/
|   |-- api_client.dart
|   |-- repository.dart
|   `-- demo_repository.dart
|-- state/
|   `-- app_state.dart
|-- screens/
|   |-- welcome_screen.dart
```

```
|   |-- login_screen.dart
|   |-- student_classes_screen.dart
|   |-- staff_classes_screen.dart
|   |-- staff_students_screen.dart
|   |-- student_grid.dart
|   |-- class_students_screen.dart
|   |-- class_tasks_screen.dart
|   |-- agenda_screen.dart
|   |-- days_screen.dart
|   |-- task_detail_screen.dart
|   |-- task_editor_screen.dart
|   |-- admin_screen.dart
|   |-- user_editor_screen.dart
|   |-- class_editor_screen.dart
|   |-- associations_screen.dart
|   |-- settings_screen.dart
|   `-- access_editor.dart
`-- ui/
    |-- ui.dart
    |-- routes.dart
    |-- theme.dart
    `-- widgets/
        |-- action_tile.dart
        |-- app_footer.dart
        |-- brand.dart
        |-- content_picture.dart
        |-- form_actions.dart
        |-- icon_action.dart
        |-- load_panel.dart
        |-- notice.dart
        |-- page_shell.dart
        |-- paged_items.dart
        |-- section_title.dart
        |-- speech_button.dart
        |-- status_badge.dart
        |-- student_picker.dart
        |-- task_badges.dart
        `-- user_avatar.dart
```

Directorio	Responsabilidad	Dependencias habituales
`models`	Representación, conversión y validación de datos.	Dart; utilidades puntuales de Flutter para el temporizador.
`services`	Contrato de acceso a datos, implementación HTTP y demostración.	Modelos, `http`, `shared_preferences`.
`state`	Sesión compartida, repositorio activo y notificaciones.	Modelos, servicios, `ChangeNotifier`.
`screens`	Composición de cada recorrido, formularios y estado local.	Modelos, `AppScope`, componentes de `ui`.

Directorio	Responsabilidad	Dependencias habituales
`ui`	Navegación, tema y controles reutilizables.	Flutter; algunos componentes acceden al estado o a plugins.

La separación es funcional: una pantalla organiza una acción, el repositorio resuelve cómo obtener o guardar sus datos y el modelo define su representación. El proyecto no utiliza BLoC, Riverpod ni una capa de controladores generados.

2.3. Contenedores de plataforma

Ubicación	Contenido significativo
`linux/CMakeLists.txt`	Nombre del binario, dependencias GTK y composición del bundle.
`linux/runner/main.cc`	Entrada nativa que crea la aplicación GTK.
`linux/runner/my_application.cc`	Ventana, tamaño inicial 1280 × 900, vista Flutter y registro de plugins.
`linux/flutter/`	Integración de compilación y registro generado de plugins.
`android/app/build.gradle.kts`	Identificador `es.clench.autovida`, SDK, destino Java 17 y configuración de compilación.
`android/settings.gradle.kts`	Plugins de Gradle y localización del SDK Flutter.
`android/app/src/main/AndroidManifest.xml`	Permiso de Internet, actividad principal y consulta del servicio TTS.
`android/app/src/debug/` y `profile/`	Permiso de tráfico HTTP en esas modalidades.
`android/app/src/main/kotlin/.../MainActivity.kt`	Actividad nativa que hereda de `FlutterActivity`.
`web/index.html`	Documento en español y carga de `flutter_bootstrap.js`.
`web/manifest.json`	Nombre, icono y presentación de la aplicación web.

La configuración Android de release conserva firma de debug. La existencia de la carpeta no equivale a un paquete de producción validado.

Las fotos de alumnos y las imágenes de contenido llegan como datos, no como rutas a recursos locales de Flutter. El `pubspec.yaml` actual no declara una carpeta de assets Dart; los iconos del contenedor web o Android se configuran en sus respectivas plataformas.

2.4. Dependencias

Paquete	Uso
`flutter`, `flutter_localizations`	Widgets Material y localización de la interfaz.
`http`	Peticiones, respuestas y subida multipart.
`intl`	Presentación de fechas y horas.
`flutter_svg`	Representación de fotos SVG predeterminadas.
`file_selector`	Selección de imágenes desde administración.

Paquete	Uso
`flutter_tts`	Lectura en voz alta mediante `SpeechButton`.
`shared_preferences`	Persistencia local de caché y operaciones pendientes.
`cupertino_icons`	Biblioteca de iconos declarada como dependencia.
`flutter_test`, `flutter_lints`	Pruebas y análisis estático.

El YAML declara el rango permitido de versiones y el lock conserva la resolución usada por la aplicación. Una actualización de plugins puede requerir recompilación nativa y comprobación de cada plataforma.

3. Arquitectura y ciclo de ejecución

3.1. Punto de entrada y composición de la aplicación

main.dart inicializa Flutter y monta el widget raíz:

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const AutovidaApp());
}
```

app.dart contiene `AutovidaApp` y su estado. Sus responsabilidades son crear o recibir `AppState`, seleccionar el inicio por rol, configurar español como idioma y aplicar tema y escala de texto.

El inicio depende de la sesión, no de una ruta de texto:

```
Widget get _inicio => switch (state.user?.role) {
  null => const WelcomeScreen(),
  UserRole.student => AgendaScreen(student: state.user!),
  UserRole.staff => const StaffClassesScreen(),
  UserRole.parent => const FamilyStudentsScreen(),
  UserRole.admin => const AdminScreen(),
};
```

La aplicación admite un `AppState` inyectado. Si lo recibe, no asume su propiedad ni lo libera al destruirse; si lo crea internamente, también lo destruye. Este criterio permite probar las mismas pantallas con repositorios controlados.

3.2. Árbol de widgets principal

```
AutovidaApp
`-- AppScope
    `-- AnimatedBuilder, suscrito a AppState
        `-- MaterialApp, clave sessionVersion
            `-- Pantalla inicial según el rol
                `-- PageShell
                    |-- Bandas de estado, si corresponden
                    |-- Cabecera
                    |-- Contenido de la pantalla
```

```
                `-- AppFooter, cuando la pantalla lo incorpora
```

`AppScope` publica el estado mediante `InheritedNotifier<AppState>`. `AppScope.of(context)` devuelve la instancia compartida y registra la dependencia del widget. No crea otra sesión ni otro repositorio.

`AnimatedBuilder` escucha las notificaciones de `AppState` y reconstruye `MaterialApp`. Mientras se conserva su clave, Flutter mantiene el estado de navegación correspondiente. Cambiar `sessionVersion` cambia esa identidad y descarta la pila de la sesión anterior.

3.3. Capas de ejecución

```
flowchart LR
    P["Pantalla"] --> S["AppState / AppScope"]
    S --> R["AutovidaRepository"]
    R --> L["LiveRepository"]
    R --> D["DemoRepository"]
    L --> H["ApiClient"]
    H --> API["API HTTP"]
```

`AppState` es el punto de acceso compartido, pero no envuelve todas las operaciones. Autenticación y refresco de perfil pasan por sus métodos; una pantalla puede llamar a `state.repository.saveTask(...)` para una operación de dominio. La comunicación HTTP sigue concentrada en los servicios.

3.4. Estado global

state/app_state.dart define `AppState`, que hereda de `ChangeNotifier`.

Miembro	Función
`repository`	Implementación de datos activa.
`user`	Usuario autenticado y preferencias cargadas.
`serverUrl`	URL inicializada mediante `API_BASE_URL`.
`student`	Indica si la sesión pertenece a un alumno.
`access`	Preferencias del usuario o valores predeterminados sin sesión.
`canEditOwnSettings`	Control de acceso a los ajustes propios.
`sessionVersion`	Identidad de la sesión y su navegación.
`avatarVersion`	Invalida la imagen cargada por `UserAvatar`.
`classroomVersion`	Invalida listas y contadores de aulas.

El login actualiza el usuario después de completar la operación del repositorio:

```
Future<void> login(String username, String password, UserRole role) async {
  user = await repository.login(username, password, role);
  notifyListeners();
}
```

`LiveRepository.login()` obtiene el token, descubre capacidades, carga el perfil completo y prepara la caché de alumno. Solo entonces entrega el usuario a `AppState`. La notificación permite seleccionar el inicio y aplicar sus preferencias.

`expire()` elimina el usuario, incrementa `sessionVersion` y notifica. `logout()` espera al repositorio y limpia la sesión cuando este confirma que ya no tiene usuario. `setDemo()` y `setServer()` sustituyen el repositorio y reinician la sesión.

El estado mantiene además un temporizador de 15 segundos para intentar enviar operaciones pendientes. `dispose()` cancela ese temporizador y cierra el repositorio.

3.5. Estado local y ciclo de vida de las pantallas

Las búsquedas, fechas, filtros, controladores de formulario, mensajes y estados de espera viven en el `State` de cada pantalla. No se almacenan globalmente porque pertenecen a una instancia del recorrido.

Método	Uso en el proyecto
`initState()`	Inicializar campos desde parámetros y crear la primera petición de un cargador.
`didChangeDependencies()`	Inicializar datos que necesitan `AppScope`; por ejemplo, copiar ajustes o comprobar la versión de una foto.
`didUpdateWidget()`	Reaccionar a un parámetro distinto sobre el mismo estado; `UserAvatar` vuelve a cargar si cambia el usuario.
`build()`	Construir widgets a partir del estado.
`dispose()`	Liberar controladores, temporizadores, cliente de datos o lectura de voz.

Después de una operación asíncrona, las pantallas comprueban `mounted` antes de actualizar estado o navegar. El usuario puede haber cerrado la ruta mientras se esperaba una respuesta.

`setState()` actualiza una pantalla. `notifyListeners()` informa a los consumidores del estado global. Repetir una consulta requiere, además, revisar cómo se conserva su `Future`.

4. Modelos y transformación de datos

4.1. Modelo de dominio

models/domain.dart reúne los objetos compartidos:

Tipo	Contenido	Uso principal
`Json`	Alias de `Map<String, dynamic>`.	Intercambio con la capa HTTP.
`UserRole`	Rol técnico y etiquetas visibles.	Inicio, permisos y selección de rutas.
`AppUser`	Usuario, nombre, rol, correo, URL de foto y accesibilidad.	Sesión, listas, editores y destinatario de agenda.
`AccessSettings`	Preferencias `stud_access_*`.	Tema, presentación y permisos de alumno.
`Classroom`	Nombre, alumnos, personal y metadatos de completitud/recuento.	Gestión y navegación por aulas.
`AgendaTask`	Datos de tarea, asignación por fecha, autor y estado.	Agenda, detalle y edición grupal.
`TaskDraft`	Datos preparados por el editor para guardar.	Serialización de una operación de escritura.

Tipo	Contenido	Uso principal
`TaskStep`	Contenido y estado de pasos, cuando existen.	Compatibilidad y descripción de tareas.

`fromJson()` adapta los nombres de intercambio a propiedades Dart. `toJson()` serializa el objeto completo cuando procede. `TaskDraft.toApiJson()` genera específicamente el cuerpo de guardado, que no equivale al JSON completo de una tarea leída.

4.2. Usuario y preferencias

`AppUser.fromJson()` convierte `full_name` en `fullName`, toma el rol de la respuesta o del argumento explícito y recopila los campos de accesibilidad conocidos. Cuando una lista omite el rol, el repositorio pasa el rol que corresponde a esa consulta.

`AccessSettings` combina valores predeterminados con los recibidos. Los accesores construyen el nombre de intercambio:

```
bool flag(String key) => values['stud_access_$key'] == true;
int number(String key) => (values['stud_access_$key'] as num).toInt();
String color(String key) => values['stud_access_$key'] as String;
AccessSettings copy() => AccessSettings(toJson());
```

Un formulario edita una copia. Así, escribir o pulsar un interruptor no modifica la sesión compartida antes de guardar. En `SettingsScreen` se conserva otra copia inicial para enviar únicamente diferencias.

4.3. Aulas: total y completitud son datos distintos

`Classroom.studentCount` utiliza el total recibido cuando existe y, en otro caso, la longitud de la lista:

```
int get studentCount => _studentCount ?? students.length;
```

`staffComplete` indica si la lista de personal es completa. Una lista vacía y una lista que no se ha podido recuperar no significan lo mismo. Las tarjetas de administración solo presentan el contador de personal cuando esa información se considera completa.

4.4. Tareas y cuerpos de guardado

`AgendaTask` conserva identificador, fecha de asignación, orden, hora, duración, creador, estado y aula de origen opcional. La pareja tarea/fecha es necesaria para consultar o retirar asignaciones individuales.

`TaskDraft` se construye desde los campos del editor. Un cuerpo básico enviado por el cliente tiene este formato:

```
{
  "task_title": "Preparar la mochila",
  "task_date": "2026-11-11",
  "task_start_hour": "09:00:00",
  "task_duration": 15,
```

```
  "alt_task_id_fk": null
}
```

El borrador puede generar contenido en `steps` a partir de la descripción. `LiveRepository.saveTask()` elimina ese campo cuando la API no lo admite; el editor también oculta entonces la descripción. En la interfaz actual no se editan los pasos por separado.

Las fechas se serializan con `isoDate()` como `AAAA-MM-DD`; las horas se envían como `HH:mm:ss`.

4.5. Validación y utilidades

Archivo	Responsabilidad
`validation.dart`	Campos obligatorios, usuario, nombre, correo, contraseña, enteros y horario.
`student_search.dart`	Filtrado por nombre/usuario sin distinguir mayúsculas ni tildes.
`task_timer.dart`	Temporizador con pausa, reanudación y cálculo por tiempo transcurrido; sin pantalla activa en esta iteración.

`passwordError()` cuenta bytes UTF-8, con máximo de 72, sin recortar espacios de la contraseña. El editor de usuarios exige cifras `0–9` para contraseñas nuevas o modificadas de alumnos, por su acceso con dibujos.

`scheduleError()` valida la duración completa: personal entre 09:00 y 15:00; familia y alumnado fuera de ese intervalo o sin hora fija. Estas reglas se aplican también en fin de semana.

5. Interfaz y componentes compartidos

5.1. Tema y escalado

ui/theme.dart concentra la identidad visual:

- `Paleta`: colores de fondo, superficie, texto, estados y controles.
- `Medidas`: tamaños, radios, trazos y ancho máximo del contenido.
- `appTheme()`: configura Material 3, tipografía y estilos de formularios, botones, tarjetas, diálogos y listas.

El tema propio del alumno utiliza sus colores y contraste. La escala de letra se calcula en `app.dart`:

```
final factorDelPerfil = state.student
    ? (state.access.number('fontsize') / 16).clamp(1.0, 2.5)
    : 1.0;
```

Este factor se multiplica por la escala del sistema en un `MediaQuery`. Ver la agenda de un alumno desde una sesión de personal no cambia el rol ni aplica automáticamente todo el tema del alumno al resto de la aplicación.

5.2. Estructura común de pantalla

`PageShell` construye una `Scaffold` con `SafeArea`, bandas de estado, cabecera y contenido desplazable. Mantiene el contenido centrado cuando cabe y permite desplazamiento cuando el

tamaño de pantalla o la escala de texto lo requieren. El pie se añade explícitamente mediante `footer`.

Una pantalla de consulta suele seguir esta composición:

```
PageShell(
  title: 'Mis aulas',
  footer: const AppFooter(),
  child: LoadPanel<List<Classroom>>(
    key: ValueKey(state.classroomVersion),
    load: state.repository.classes,
    builder: (aulas) => PagedItems<Classroom>(
      items: aulas,
      builder: (aula) => Text(aula.name),
    ),
  ),
)
```

El ejemplo muestra el patrón de composición. Cada pantalla incorpora sus tarjetas, filtros y claves concretas.

5.3. Carga asíncrona: `LoadPanel`

`LoadPanel<T>` recibe una función `load` y una función `builder`. Conserva la petición en su estado:

```
late Future<T> peticion;

@override
void initState() {
  super.initState();
  peticion = widget.load();
}

void reintentar() => setState(() => peticion = widget.load());
```

Un `FutureBuilder` presenta espera, error con reintento o el contenido. Mantener el `Future` evita repetir la consulta en cada reconstrucción. Como consecuencia, cambiar la función `load` sobre el mismo widget no repite automáticamente la petición: para una recarga se cambia su clave o se solicita reintento.

5.4. Paginación y búsqueda

`PagedItems<T>` pagina una colección ya cargada. Calcula el número de páginas, limita el índice si cambia el tamaño y dibuja lista o rejilla. No solicita páginas a la API.

Las flechas se sitúan a los lados con ancho suficiente y debajo en pantallas estrechas. Si solo existe una página, no se muestran controles de navegación.

`StaffStudentsScreen` aplica `filterStudents()` antes de construir la lista y usa el texto de búsqueda como clave del contenedor. Cambiar el criterio vuelve a la primera página sin perder resultados que antes estuvieran en páginas posteriores.

5.5. Catálogo de componentes

Archivo	Función
`action_tile.dart`	Ficha navegable horizontal o vertical, con icono/foto y acción secundaria opcional.
`app_footer.dart`	Inicio, ajustes permitidos y cierre de sesión.
`brand.dart`	Marca visual compartida.
`content_picture.dart`	Imagen o icono de una tarea; `taskIcon()` selecciona icono según el título.
`form_actions.dart`	Acciones guardar/cancelar y diálogo `confirmDelete()`.
`icon_action.dart`	Acción compacta con etiqueta accesible.
`load_panel.dart`	Resolución visual de una petición asíncrona.
`notice.dart`	Aviso informativo/error con región semántica activa.
`page_shell.dart`	Estructura de pantalla y banda de sincronización.
`paged_items.dart`	Lista o rejilla paginada localmente.
`section_title.dart`	Encabezado de sección.
`speech_button.dart`	Lectura TTS en español y mensaje si la plataforma no ofrece voz.
`status_badge.dart`	Insignias de estado y rol.
`student_picker.dart`	Selector reutilizable preparado para búsqueda y selección; no se monta en la vista actual de personal.
`task_badges.dart`	Composición de insignias de tarea.
`user_avatar.dart`	Carga y representación de foto, SVG o silueta alternativa.

`ui/ui.dart` reexporta navegación, tema y componentes para simplificar las importaciones de las pantallas.

5.6. Ajustes, fotos y accesibilidad

Ajuste `stud_access_*`	Aplicación en el cliente
`main_col`, `sec_col`, `contrast`	Colores y contraste de la sesión de alumno.
`fontsize`	Escala de texto.
`ntasks_visible`	Cantidad de tareas por página; cero muestra todas.
`ndays_visible`	Selección de días; cero oculta el acceso a elegir día en la agenda propia.
`pictograms`	Apoyos visuales de tareas.
`filters`	Filtros de estado en agenda propia.
`talkback`	Botones de lectura de texto.
`profile_mod`	Acceso y edición de ajustes propios.
`task_mod`	Creación y eliminación de tareas propias.
`prof_mod`, `timebar`	Se conservan en el modelo; sin un recorrido de interfaz específico activo.

`AccessEditor` modifica la copia de preferencias recibida. Con `permissions: true`, incorpora los interruptores administrativos. El alumno no recibe esos controles ni campos para cambiar contraseña o foto en sus ajustes.

`UserAvatar` carga mediante el repositorio y conserva el `Future` de imagen. Comprueba `avatarVersion` en `didChangeDependencies()` y el usuario en `didUpdateWidget()`. Si falta la foto o falla su carga, representa una silueta. Las imágenes SVG se procesan con `flutter_svg`.

La subida de foto utiliza `file_selector`, limita el tamaño a 5 MB y envía el archivo con multipart. `SpeechButton` utiliza `flutter_tts`, selecciona español y detiene la lectura al destruirse.

Los controles compartidos incorporan tamaños amplios, etiquetas, foco y estados con texto. Las comprobaciones automáticas de accesibilidad cubren casos concretos; la validación con dispositivos de apoyo sigue siendo una actividad separada.

6. Integración HTTP utilizada por el frontend

6.1. Contrato del repositorio y cliente HTTP

services/repository.dart contiene `AutovidaRepository`, las acciones derivadas y `LiveRepository`. services/demo_repository.dart implementa el mismo contrato con datos en memoria.

`LiveRepository` selecciona rutas, transforma respuestas en modelos y mantiene cachés. services/api_client.dart prepara peticiones, Bearer, JSON UTF-8, timeout de 20 segundos, respuestas binarias y subida multipart.

La petición utiliza `Accept:application/json`; añade `Content-Type` cuando hay cuerpo JSON y `Authorization` cuando la operación es protegida y existe token. Los identificadores de ruta se codifican mediante `segment()`.

La respuesta normal se interpreta mediante `errorCode` y `details`:

```
{"errorCode":0,"details":"successful","classes":[]}
```

`ApiException` conserva mensaje, código y estado HTTP. El código 1002 elimina el token y activa `AppState.expire()`. El 1001 se presenta como error de credenciales en el login. Las imágenes se reciben como bytes; OpenAPI no utiliza esa envoltura JSON.

6.2. Sesión y detección de capacidades

El login encadena las siguientes operaciones:

1. Obtener el token y el usuario básico.
2. Consultar OpenAPI con `discover()`.
3. Cargar el perfil completo cuando la capacidad está disponible.
4. Preparar la caché correspondiente a la sesión.

`profile()` admite el formato plano actual y una respuesta con `user` anidado:

```
final data = await api.request('GET', '/users/me');
return _user = AppUser.fromJson(
  Map<String, dynamic>.from(data['user'] as Map? ?? data),
);
```

`supports()` comprueba método y ruta. `camposDelCuerpo()` revisa las propiedades admitidas por la petición. Las pantallas utilizan capacidades como `supportsTaskContent`, `canReadAgenda`, `canTrackProgress` o `canSetAvatarOf` para presentar acciones compatibles.

6.3. Endpoints de sesión y perfil

Las tablas siguientes relacionan las peticiones utilizadas con el método del cliente y el recorrido que las origina. No incluyen el catálogo administrativo completo de la API ni rutas sin uso en las pantallas actuales.

Método y ruta	Método del cliente	Uso
`POST/auth/login`	`login()`	Acceso por rol desde `LoginScreen`.
`POST/auth/logout`	`logout()`	Salida desde `AppFooter`.
`GET/openapi.json`	`discover()`	Detección de capacidades al autenticar.
`GET/users/me`	`profile()`	Perfil tras el login y refresco de ajustes.
`GET/`	`_synchronize()`	Comprobación de recuperación de conexión; también se utiliza en el diagnóstico del entorno.

El token permanece en memoria. Reiniciar la aplicación exige autenticar de nuevo. Un cambio de contraseña o de identificador de la cuenta propia puede invalidar la sesión.

6.4. Endpoints de usuarios y fotos

Método y ruta	Método del cliente	Pantalla o componente
`GET/users`	`users()`	Lista administrativa y selección de asociaciones.
`POST/users`	`saveUser()` sin `previous`	Alta desde `UserEditorScreen`.
`PUT/users/{role}/{username}`	`saveUser()` con `previous`	Edición administrativa y `Settings Screen`.
`DELETE/users/{role}/ {username}`	`deleteUser()`	Eliminación desde `AdminScreen`.
`PUT/users/students/{student_ username}/avatar`	`uploadAvatarOf()`	Cambio de foto desde `UserEditorScreen`.
`GET/users/students/me/avatar`	`avatar()`	Foto propia de la sesión de alumno.
`GET/users/students/{student_ username}/avatar-image`	`avatar()`	Fotos de selección de alumno o de alumnos consultados.

La administración usa las operaciones de cuentas y foto ajena. Los ajustes propios envían únicamente los campos permitidos para el usuario autenticado. La subida de foto utiliza `multipart/ form-data` con campo `file`.

La carga de fotos ajenas está sujeta a autorización. Una familia autenticada puede recibir un rechazo en esa ruta y el componente mostrará la silueta; esa representación no demuestra que la imagen no exista.

6.5. Endpoints de aulas y miembros

Método y ruta	Método del cliente	Pantalla o recorrido
`GET/classes`	`directory()`, `classes()`, `unassignedStudents()`	Directorio previo al login, lista de aulas y candidatos administrativos.

Método y ruta	Método del cliente	Pantalla o recorrido
`GET/classes/{class_name}`	`_aulaPublica()`, `classDetail()`	Selección de alumno, consulta y edición de aula.
`POST/classes`	`saveClass()` sin nombre anterior	Crear aula desde administración.
`PUT/classes/{current_class_name}`	`saveClass()` con nombre anterior	Renombrar aula.
`DELETE/classes/{class_name}`	`deleteClass()`	Eliminar aula desde administración.
`POST/classes/{class_name}/staff/{staff_username}`	`addStaff()`	Asignación desde editor de aula o asociaciones.
`DELETE/classes/{class_name}/staff/{staff_username}`	`removeStaff()`	Retirada desde editor de aula.
`POST/classes/{class_name}/students/{student_username}`	`enroll()`	Alta de alumno desde administración.
`DELETE/classes/{class_name}/students/{student_username}`	`enroll(remove: true)`	Retirada desde administración o personal del aula.

`directory()` consulta aulas y alumnado sin Bearer para el acceso visual. `classes()` utiliza la sesión y obtiene las aulas que esta puede consultar.

El cliente bloquea el alta desde personal antes de enviar la petición. La capacidad `canEnrollStudents` se utiliza también para mostrar la gestión de alumnos existentes; su nombre no debe interpretarse como autorización suficiente para crear una matrícula.

`unassignedStudents()` combina las cuentas y las matrículas para excluir a los alumnos que ya tienen aula. El editor administrativo es su consumidor activo. El personal no utiliza ese selector en su recorrido actual.

6.6. Endpoints de asociaciones

Método y ruta	Método del cliente	Uso
`GET/student-links/parents/{parent_username}/students`	`linkedStudents()`	Alumnos de una familia.
`GET/student-links/staff/{staff_username}/students`	`linkedStudents()`	Alumnos del personal y consulta administrativa de sus asociaciones.
`GET/student-links/students/{student_username}`	`parents()`, `_classroomWithStaff()`	Asociaciones del alumno y recuperación del personal del aula.
`POST/student-links/students/{student_username}/parents/{parent_username}`	`linkParent()`	Crear asociación familiar desde administración.
`DELETE/student-links/students/{student_username}/parents/{parent_username}`	`linkParent(remove: true)`	Retirar asociación familiar.

La pantalla de asociaciones del personal asigna un aula mediante `addStaff()`. Su lista de alumnos se obtiene después con `linkedStudents()`.

6.7. Endpoints de agenda y tareas

Método y ruta	Método del cliente	Uso
`GET/student-tasks/me?task_date=AAAA-MM-DD`	`agenda()`	Agenda propia del alumno.
`GET/student-tasks/students/{student_username}?task_date=AAAA-MM-DD`	`agenda()`	Agenda del alumno consultada por personal/familia.
`POST/student-tasks/me`	`saveTask()`	Nueva tarea propia con permiso.
`POST/student-tasks/students/{student_username}`	`saveTask()`	Nueva tarea individual por personal/familia.
`DELETE/student-tasks/me/{task_id}?task_date=AAAA-MM-DD`	`deleteTask()`	Eliminar tarea propia permitida.
`DELETE/student-tasks/students/{student_username}/{task_id}?task_date=AAAA-MM-DD`	`deleteTask()`	Retirar tarea individual del alumno asociado.
`GET/classes/{class_name}/tasks`	`classTasks()`	Listado de tareas grupales.
`POST/classes/{class_name}/tasks`	`saveTask(classroom:...)`	Crear tarea grupal.
`PUT/classes/{class_name}/tasks/{task_id}`	`saveTask(classroom:...,previous:...)`	Editar tarea grupal.
`DELETE/classes/{class_name}/tasks/{task_id}`	`deleteTask(classroom:...)`	Eliminar tarea grupal.

Las operaciones individuales incluyen la fecha de asignación cuando se indica. `saveTask()` decide el destino utilizando la sesión, la presencia de `classroom` y la tarea previa.

La pantalla de detalle incorpora además una acción condicional de progreso:

Método y ruta	Método del cliente	Disponibilidad
`PATCH/student-tasks/me/{task_id}/progress?task_date=AAAA-MM-DD`	`completeTask()` → `progress()`	Solo se ofrece si la API anuncia la capacidad. No está disponible en la API de referencia.

Esta condición permite que una misma pantalla funcione con demostración o con una API que admita progreso, sin presentar esa acción en una conexión que no lo soporte.

7. Navegación y flujo entre pantallas

7.1. Navegación tipada por resultados

ui/routes.dart utiliza `Navigator` y `MaterialPageRoute`:

```
Future<T?> openPage<T>(BuildContext context, Widget page) =>
    Navigator.of(context).push<T>(MaterialPageRoute(builder: (_) => page));

void goHome(BuildContext context) =>
    Navigator.of(context).popUntil((route) => route.isFirst);
```

Los parámetros entran por el constructor del widget. El resultado vuelve mediante `Navigator.pop(context,resultado)`. No se utiliza un router declarativo ni una variable global de alumno seleccionado.

El patrón de apertura de un editor es:

```
final guardada = await openPage<bool>(
  context,
  TaskEditorScreen(student: widget.student, date: dia),
);
if (mounted && guardada == true) {
  _recargar(mensaje: 'La tarea se ha guardado.');
}
```

El editor devuelve `true` después de guardar. El cierre mediante «Volver» o «Cancelar» suele devolver `null`. Algunas pantallas recargan con cualquier resultado porque pueden contener acciones ya guardadas antes de cerrarse.

Los identificadores que comienzan por _ son privados a la biblioteca Dart. Los estados `_...State` y los widgets auxiliares privados agrupan detalles de implementación de la pantalla; no son rutas adicionales.

7.2. Mapa de recorridos

```
flowchart TD
    W[WelcomeScreen] --> SC[StudentClassesScreen]
    SC --> SS[StudentSelectionScreen]
    SS --> LS[LoginScreen con alumno]
    SC --> LF[LoginScreen sin foto]
    W --> LC[LoginScreen del centro]
    LS --> A[AgendaScreen]
    LF --> A
    LC --> ST[StaffClassesScreen]
    LC --> F[FamilyStudentsScreen]
    LC --> AD[AdminScreen]
    ST --> CS[ClassStudentsScreen]
    ST --> SA[StaffStudentsScreen]
    CS --> A
    SA --> A
    F --> A
    CS --> CT[ClassTasksScreen]
    CT --> TE[TaskEditorScreen]
    A --> DS[DaysScreen]
    A --> TD[TaskDetailScreen]
    A --> TE
    AD --> UE[UserEditorScreen]
    AD --> CE[ClassEditorScreen]
    UE --> AS[AssociationsScreen]
```

`SettingsScreen` se abre desde `AppFooter` cuando la sesión dispone de permiso. Las conexiones del diagrama representan recorridos de usuario; la pantalla inicial después del login la selecciona `app.dart`.

7.3. Responsabilidad de cada archivo de pantallas

Archivo	Clases principales	Entrada y resultado
`welcome_screen.dart`	`WelcomeScreen`	Abre acceso del alumno o login del centro.
`login_screen.dart`	`LoginScreen`	Recibe roles permitidos y alumno opcional; autentica y vuelve al inicio.
`student_classes_screen.dart`	`StudentClassesScreen`, `StudentSelectionScreen`	Carga el directorio y pasa el aula y alumno elegidos.
`staff_classes_screen.dart`	`StaffClassesScreen`	Lista aulas de la sesión; pasa `className` al detalle.
`staff_students_screen.dart`	`StaffStudentsScreen`, `FamilyStudentsScreen`	Obtienen alumnos asociados y los pasan a la agenda.
`student_grid.dart`	`StudentGrid`	Presenta alumnos en lista/rejilla y abre su agenda.
`class_students_screen.dart`	`ClassStudentsScreen`	Consulta un aula y permite gestionar sus alumnos existentes.
`class_tasks_screen.dart`	`ClassTasksScreen`	Recibe `Classroom`, lista tareas grupales y abre el editor.
`agenda_screen.dart`	`AgendaScreen`	Recibe `AppUser`; mantiene fecha, filtros y revisión de tareas.
`days_screen.dart`	`DaysScreen`	Recibe fecha y devuelve un `DateTime`.
`task_detail_screen.dart`	`TaskDetailScreen`	Recibe alumno y tarea; consulta, elimina o registra progreso si está disponible.
`task_editor_screen.dart`	`TaskEditorScreen`	Recibe alumno, fecha, tarea opcional y aula opcional; devuelve `true` al guardar/eliminar.
`admin_screen.dart`	`AdminScreen`	Alterna usuarios y aulas y abre sus editores.
`user_editor_screen.dart`	`UserEditorScreen`	Recibe usuario opcional; crea/edita cuenta, foto y acceso a asociaciones.
`class_editor_screen.dart`	`ClassEditorScreen`	Recibe aula opcional; crea/renombra y gestiona sus miembros.
`associations_screen.dart`	`AssociationsScreen`	Recibe usuario y guarda asociaciones familiares o asignación a aula.
`settings_screen.dart`	`SettingsScreen`	Edita los ajustes propios y actualiza el perfil compartido.
`access_editor.dart`	`AccessEditor`	Formulario reutilizable sobre una copia de `AccessSettings`.

7.4. Login visual del alumno

1. `StudentClassesScreen` llama a `directory()` y presenta aulas.
2. La selección pasa un `Classroom` a `StudentSelectionScreen`.
3. La foto pulsada pasa un `AppUser` a `LoginScreen(student:...)`.
4. Login fija usuario y rol; los dibujos introducen la contraseña numérica.
5. `AppState.login()` recibe el perfil completo del repositorio y notifica.

6. La agenda se convierte en inicio de sesión y `goHome()` retira las pantallas de acceso.

Si no se carga el directorio, se ofrece escribir el nombre de usuario. La contraseña del alumno sigue utilizando dibujos.

La correspondencia cifra/dibujo se define por el orden de `_dibujos` en `login_screen.dart`:

Cifra	Dibujo	Cifra	Dibujo
0	Sol	5	Coche
1	Corazón	6	Pelota
2	Estrella	7	Casa
3	Animal	8	Árbol
4	Flor	9	Música

El marcador representa la longitud introducida; `_escribir()` añade una cifra y `_borrar()` elimina la última. El orden de la tabla forma parte del significado de las contraseñas existentes.

7.5. Sesión autenticada y alumno consultado

`AppState.user` identifica a quien ha iniciado sesión. `widget.student` identifica al destinatario de una agenda. Un miembro del personal puede abrir `AgendaScreen(student: alumno)` manteniendo su propia sesión.

El repositorio utiliza el rol autenticado para elegir entre agenda propia y agenda asociada. Esta distinción debe mantenerse en nuevas pantallas: el rol del destinatario no determina las credenciales de la petición.

`StaffStudentsScreen` y `FamilyStudentsScreen` cargan `linkedStudents()`. `StudentGrid` abre la agenda del alumno pulsado y ejecuta su callback `onReturn` al regresar, cuando la pantalla superior necesita recargar la lista.

7.6. Agenda, fecha y tareas

`AgendaScreen` mantiene `dia`, `filtro` y `revision`. La clave de la consulta combina revisión y fecha. El filtro de estado se aplica a la colección recibida y el número de tareas por página depende de la accesibilidad.

`DaysScreen` solo devuelve una fecha; no guarda información remota. La agenda actualiza `dia`, cambia la revisión y vuelve a cargar las tareas.

Al crear una tarea se abre `TaskEditorScreen`. El formulario valida, prepara `TaskDraft` y llama al repositorio. Al volver con `true`, la agenda repite su consulta. Al abrir un detalle, la agenda se recarga al regresar con cualquier resultado, porque puede haberse guardado una acción sin cerrar inmediatamente la pantalla.

```
sequenceDiagram
    participant A as AgendaScreen
    participant E as TaskEditorScreen
    participant R as Repository
    participant H as ApiClient
    A->>E: alumno y fecha
    E->>E: validar y crear TaskDraft
    E->>R: saveTask
    R->>H: petición con cuerpo compatible
```

```
    H-->>R: respuesta confirmada
    R-->>E: AgendaTask
    E-->>A: Navigator.pop(true)
    A->>R: agenda del alumno y fecha
    R-->>A: lista actualizada
```

En `TaskDetailScreen`, la tarea recibida se copia a una variable de estado local. La pantalla puede actualizar esa copia tras una confirmación de progreso. La eliminación pide confirmación y devuelve `true` cuando termina correctamente.

Las tareas individuales creadas por un alumno solo se ofrecen para eliminar si son suyas y tiene `task_mod`. Las tareas de aula se gestionan desde el recorrido grupal.

El indicador de aula de la agenda utiliza `classOf()`. Su `Future` se crea en el `build` del widget auxiliar `_ClaseDelAlumno`; una reconstrucción puede volver a consultar el directorio. Si se amplía ese componente, conviene considerar esa frecuencia de carga.

7.7. Tareas de aula

`ClassStudentsScreen` pasa el aula cargada a `ClassTasksScreen`. Esta lista consulta las tareas grupales y abre el editor con `classroom` y un alumno de referencia.

Ese alumno permite reutilizar el formulario, pero la escritura va a la ruta de tareas de aula. La operación no se convierte en una tarea individual del alumno de referencia. Si el aula está vacía, el recorrido no permite crear tareas.

La edición grupal devuelve `true` y actualiza la revisión de la lista. El aula recibida por constructor es una instantánea; una ampliación que permita cambiar miembros desde esa pantalla tendrá que refrescar también ese contexto.

7.8. Administración y guardados independientes

`AdminScreen` mantiene búsqueda, filtro de rol, vista de usuarios/aulas y revisión. Los editores reciben el objeto seleccionado, o `null` para crear uno nuevo.

`UserEditorScreen` guarda los campos del formulario mediante `saveUser()`. El cambio de foto y las asociaciones son operaciones independientes que se guardan cuando se pulsa su acción.

`ClassEditorScreen` mantiene `nombreGuardado` para distinguir un aula existente. Al crearla, actualiza ese nombre y habilita la gestión de miembros. Las altas/bajas de miembros se guardan de inmediato y actualizan `classroomVersion`.

Por tanto, «Cancelar» cierra un formulario, pero no deshace fotos, matrículas o asociaciones que ya se hayan confirmado. Administración vuelve a consultar sus listas al regresar de un editor incluso cuando este devuelve `null`.

7.9. Ajustes propios

`SettingsScreen` inicializa una copia editable y otra original de las preferencias en `didChangeDependencies()`, una sola vez por instancia. El guardado compara ambas y envía únicamente cambios.

Después de guardar llama a `refreshProfile()`. La notificación de `AppState` reconstruye tema y escala con la respuesta actual. El acceso de alumno se controla desde `canEditOwnSettings`; el resto de perfiles utiliza esa pantalla para cambiar su contraseña.

8. Actualización de datos y persistencia

8.1. Invalidación de consultas

Las claves de widgets identifican el estado que Flutter conserva. En una consulta, variar la clave de `LoadPanel` crea otra instancia de su estado y ejecuta de nuevo `load`.

Ejemplo de la lista administrativa:

```
key: ValueKey('aulas/$revision/${state.classroomVersion}'),
```

`revision` cubre cambios locales y `classroomVersion` comunica modificaciones compartidas. El resultado de una operación se confirma antes de invalidar la lista.

Cambio	Actualización utilizada
Guardado o borrado de tarea	Revisión de agenda o tareas de aula.
Fecha seleccionada	Fecha y revisión en la clave del cargador.
Matrícula o personal asignado	`classroomsChanged()` y revisión del editor/detalle.
Regreso a aulas del personal	Nueva revisión local.
Regreso a administración	Nueva consulta de listas.
Foto guardada	`photoChanged()` y `avatarVersion`.
Preferencias propias	`refreshProfile()` y notificación global.

El total se obtiene de `Classroom`, no se incrementa visualmente antes de guardar. Las notificaciones son internas a la aplicación; no constituyen una suscripción a cambios de otros dispositivos.

8.2. Información derivada del repositorio

`classOf()` busca el aula de un alumno en el directorio. `unassignedStudents()` prepara candidatos administrativos restando los alumnos ya matriculados. Ambas son composiciones de lecturas existentes y no requieren un endpoint específico nuevo.

`_classroomWithStaff()` completa la información del personal cuando no viene en la respuesta de aula. En administración puede obtenerla a través de las asociaciones de uno de sus alumnos. En aulas vacías solo conserva operaciones confirmadas durante la sesión, marca la lista incompleta y la interfaz muestra un aviso.

Los mapas `_knownUsers` y `_knownClassStaff` viven en memoria. Sus datos se limpian al cambiar de sesión y no representan persistencia independiente.

8.3. Caché y cola

`LiveRepository` mantiene `_agendaCache` por alumno/fecha y `_pending` para confirmaciones de progreso. En sesiones de alumno, `shared_preferences` conserva esos datos bajo una clave que incluye servidor y usuario.

La lectura de agenda intenta sincronizar primero. Puede devolver una entrada ya cargada ante errores sin código ni estado HTTP; un rechazo de permisos no se reemplaza por caché.

`synchronize()` comparte el `Future` en curso mediante `_syncing` para evitar envíos simultáneos. Las operaciones confirmadas se retiran de la cola y la persistencia se actualiza después de cada envío. El temporizador global intenta enviar pendientes cada 15 segundos y `SyncBanner` ofrece un reintento manual.

Logout intenta enviar pendientes antes de eliminar la sesión. Si no puede, conserva la sesión y presenta el error. Al cerrar correctamente elimina la caché correspondiente. El token no se persiste, de modo que tras reiniciar sigue siendo necesario autenticar.

El progreso solo se utiliza si la conexión admite esa capacidad. Su presencia en la implementación y en pruebas simuladas no habilita el completado en la API de referencia.

8.4. Demostración

`DemoRepository` contiene mapas de usuarios, contraseñas, matrículas, personal, familias, tareas y fotos. Implementa las operaciones del contrato sin HTTP y mantiene la política de impedir nuevas matrículas al personal.

Los cambios permanecen mientras viva esa instancia. Logout solo cierra la sesión; reiniciar o sustituir el repositorio vuelve a crear los datos. La demostración se activa con `DEMO=true` o por inyección en pruebas y se identifica con una banda visible.

Una pérdida de conexión en modo real no activa demo. Tampoco equivale a demo conectarse a una base de datos con un nombre que contenga esa palabra.

9. Pruebas y mantenimiento

9.1. Organización de pruebas

Archivo en `test/`	Cobertura principal
`domain_test.dart`	Recuentos, formato de tareas, contraseñas, horario y temporizador.
`api_client_test.dart`	HTTP simulado, Bearer, URI, errores, caché y cola.
`main_contract_test.dart`	Contratos consumidos, perfil, ajustes, gestión administrativa y bloqueo del alta de personal.
`screen_flow_test.dart`	Acceso por roles y recorridos de pantalla con demo.
`staff_student_search_test.dart`	Búsqueda, paginación y gestión de alumnos existentes con actualización del total.
`limited_server_test.dart`	Presentación cuando se anuncian capacidades reducidas.
`sin_scroll_test.dart`	Ajuste de recorridos objetivo a 1280 × 900.
`accessibility_test.dart`	Contraste, semántica, tamaño de controles y texto ampliado en los casos cubiertos.
`ajustes_del_alumno_test.dart`	Conversión de preferencias desde la respuesta de perfil.

Las pruebas de widgets construyen `AppState` con un repositorio controlado y lo inyectan en `AutovidaApp`. Al terminar, desmontan el árbol y liberan el estado para cancelar temporizadores y otros recursos.

Las pruebas HTTP utilizan `MockClient`; no necesitan un servicio real. Las respuestas de prueba con texto acentuado deben declarar JSON UTF-8 de forma coherente con el decodificador.

9.2. Comandos

Desde `src/frontend`:

```
flutter analyze
flutter test
flutter test test/main_contract_test.dart
flutter test test/staff_student_search_test.dart
```

Compilaciones:

```
flutter build web
flutter build linux
```

La salida web está en `build/web`. Para Linux se utiliza la carpeta `bundle` con sus recursos. Una compilación no arranca el servicio ni modifica las condiciones de conexión del destino.

La comprobación de código registrada en esta revisión terminó con análisis limpio y 67 pruebas superadas. Se verificó también la sintaxis del lanzador con `bash -n`; no se ejecutó el arranque completo para redactar la documentación.

9.3. Criterio para una modificación

Una modificación visual suele afectar a `screens`, `ui` o ambos. Una modificación de datos puede afectar también al modelo y al contrato del repositorio. El recorrido recomendado es:

1. Identificar la pantalla y la acción que cambian.
2. Revisar el modelo y el método de repositorio utilizados.
3. Mantener coherentes las implementaciones real y de demostración.
4. Resolver espera, error y resultado de navegación.
5. Actualizar la consulta o la versión compartida después de confirmar la operación.
6. Ejecutar la prueba del recorrido y las comprobaciones que correspondan.

Tipo de cambio	Archivos de referencia
Identidad visual común	`ui/theme.dart`.
Acceso y teclado visual	`login_screen.dart`, `student_classes_screen.dart`.
Presentación de agenda	`agenda_screen.dart`, `days_screen.dart`, `student_grid.dart`.
Formato o validación de tarea	`domain.dart`, `validation.dart`, `task_editor_screen.dart`.
Sesión y notificaciones	`state/app_state.dart`.
Comunicación y errores	`api_client.dart`, `repository.dart`.
Datos de demo	`demo_repository.dart`.
Contadores o fotos desactualizados	Claves de carga, `classroomVersion`, `avatarVersion`.

La guía y las pruebas deben actualizarse cuando cambie un contrato consumido o un recorrido. El README conserva referencias históricas; para el comportamiento actual se deben utilizar los archivos y relaciones descritos aquí.

2.3. Documentos comunes al frontend y backend

2.3.1. Guía para lanzar el sistema (backend + frontend)

AUTOVIDA: guía de ejecución y resolución de errores

Entorno: Linux · **Rama:** `iteracion1` · **Revisión:** `11/11/2026`

Esta guía explica cómo preparar el equipo, lanzar la aplicación con `scripts/levantar_backend.sh`, ejecutar Flutter durante el desarrollo y resolver los fallos habituales. Los comandos parten de la raíz del repositorio salvo que se indique otra ubicación.

La estructura y el funcionamiento del código Flutter se documentan en la guía técnica del frontend.

Índice

- 1. Preparación y ejecución . . . 61
  - 1.1. Requisitos y dependencias . . . 61
  - 1.2. Configuración necesaria para el lanzador . . . 62
  - 1.3. Lanzar el entorno completo . . . 63
  - 1.4. Ejecución recomendada para desarrollar Flutter . . . 63
  - 1.5. Demostración y cuentas de prueba . . . 64
  - 1.6. Destinos web y Android . . . 64
  - 1.7. Parada . . . 65
- 2. Diagnóstico de errores . . . 65
  - 2.1. Arranque y herramientas . . . 65
  - 2.2. Conexión HTTP . . . 67
  - 2.3. Respuestas rechazadas . . . 67
  - 2.4. Información antigua o acciones ausentes . . . 68

1. Preparación y ejecución

1.1. Requisitos y dependencias

El lanzador está preparado para Bash en Linux. Para trabajar sobre este checkout se necesita:

Requisito	Comprobación
Git y rama `iteracion1`	`git status --short --branch`
Flutter compatible con el lock	`flutter --version`
Python 3.12 o superior y venv	`python3 --version`
Docker y plugin Compose	`docker info` y `docker compose version`

Requisito	Comprobación
Entorno Linux gráfico y dependencias nativas	`flutter doctor -v`
Chrome para el destino web	`flutter devices`

El lock actual requiere Dart `>=3.13.0` y Flutter `>=3.47.0`, aunque el YAML declare un mínimo de Dart inferior. El entorno de referencia utiliza Flutter 3.47.1 y Dart 3.13.1.

En un Ubuntu/Debian compatible, las dependencias de escritorio habituales son:

```
sudo apt install python3-venv python3-pip clang cmake ninja-build pkg-config libgtk-3-dev
```

Docker, Compose y Flutter deben estar instalados y accesibles para el usuario de desarrollo. El proyecto se ejecuta sin sudo; los permisos que pueda pedir Docker se resuelven por separado.

Desde la raíz:

```
git switch iteracion1
git status --short --branch
cd src/frontend
flutter pub get
flutter doctor -v
flutter devices
cd ../..
```

Antes de cambiar de rama deben revisarse los cambios locales. El desarrollo descrito no requiere modificar `main`.

1.2. Configuración necesaria para el lanzador

La configuración local del servicio se guarda en `src/backend/proyectFastAPI/.env` y no se versiona. Para una instalación nueva:

```
DB_USER=autovida_admin
DB_PASSWORD='SUSTITUIR_POR_LA_CLAVE_ACORDADA'
DB_HOST=localhost
DB_PORT=5432
DB_NAME=autovida_db
```

Si existe una configuración y datos previos, deben mantenerse valores compatibles con ese entorno. El script lee el archivo también como shell: sin espacios alrededor de = y con valores entrecomillados cuando corresponda.

El lanzador carga y exporta automáticamente las variables del `.env` antes de iniciar los servicios. La API y su proceso de recarga heredan esa configuración; no es necesario ejecutar `source` ni exportar variables manualmente en la terminal.

El lanzador crea el archivo si falta, pero se detiene si no contiene contraseña. Cuando el archivo ya existe con la clave vacía, hay que editarlo; el script no lo completa a partir de una variable antepuesta al comando.

1.3. Lanzar el entorno completo

El archivo real es scripts/levantar_backend.sh. Desde la raíz:

```
bash ./scripts/levantar_backend.sh --con-web
```

Opción	Resultado
Sin argumentos	Prepara el servicio y deja la API en primer plano.
`-con-web`	Añade Flutter en segundo plano; prefiere escritorio Linux y usa Chrome como alternativa.
`-parar`	Detiene el contenedor de PostgreSQL conservando su volumen.

El nombre informal `./levantar_backend` corresponde a ese archivo. El script solo comprueba el primer argumento y no implementa selector de dispositivo ni de puerto.

Sus fases son comprobación de herramientas, configuración local, preparación de `src/backend/.venv`, arranque del contenedor, preparación de datos de prueba y ejecución de la API. Con `-con-web` lanza Flutter antes de dejar la API en primer plano.

Puertos utilizados: servicio HTTP `8000`, frontend web `5174` y contenedor de datos `5432`. La comprobación inicial de escritorio es parcial; `flutter doctor` y la compilación pueden detectar requisitos adicionales.

El script no instala Flutter. Si no lo encuentra, avisa y continúa levantando el servicio. La creación de datos de prueba se omite cuando ya existen cuentas, por lo que volver a ejecutar el script no restablece contraseñas ni genera de nuevo tareas para hoy.

El arranque está completo cuando la terminal confirma que la API ha terminado de inicializarse y Flutter muestra la portada. Como ambos procesos arrancan por separado, la ventana puede abrirse antes de que la API esté lista. Si la primera consulta falla, esperar a la confirmación del servicio y utilizar «Volver a intentar».

Desde otra terminal se puede comprobar la conexión:

```
curl --fail http://127.0.0.1:8000/
```

La respuesta debe ser JSON con `errorCode: 0`. Esta comprobación valida el acceso HTTP; el login con una cuenta de prueba permite comprobar además la sesión y la carga de su pantalla inicial. El modo real no muestra la banda de demostración.

1.4. Ejecución recomendada para desarrollar Flutter

Terminal 1, desde la raíz:

```
bash ./scripts/levantar_backend.sh
```

Terminal 2, desde la raíz:

```
cd src/frontend
flutter run -d linux --dart-define=API_BASE_URL=http://127.0.0.1:8000
```

En la terminal de Flutter, `r` ejecuta hot reload, `R` reinicia el estado Dart y `q` termina la ejecución. Los cambios de dependencias, configuración nativa o `--dart-define` requieren volver a ejecutar el comando.

La URL procede de `AppState.serverUrl`. Los métodos `setServer()` y `setDemo()` permiten reemplazar el repositorio desde código, pero la portada actual no contiene una pantalla de configuración de conexión.

1.5. Demostración y cuentas de prueba

```
cd src/frontend
flutter run -d linux --dart-define=DEMO=true
```

La portada actual no ofrece un botón para activar demo. La activación se hace con ese parámetro o inyectando el repositorio en una prueba.

Cuenta de prueba	Rol	Contraseña
`admin`	Administrador	`1234`
`lucia`	Personal	`1234`
`familia`	Tutor familiar	`1234`
`pablo`, `ana`	Alumno	`1234`

Estas credenciales corresponden a la demostración y a las cuentas creadas por la semilla del lanzador, mientras no se hayan cambiado. El acceso de alumno con `1234` es Corazón → Estrella → Animal → Flor. Las tareas sembradas corresponden a la fecha de su creación.

1.6. Destinos web y Android

El script utiliza un Chrome de pruebas con perfil separado cuando no puede ejecutar escritorio. La API de referencia no declara CORS, por lo que un navegador normal puede bloquear sus peticiones aunque dibuje correctamente la interfaz.

La alternativa local del script equivale a:

```
cd src/frontend
flutter run -d chrome --web-port=5174 \
  --dart-define=API_BASE_URL=http://127.0.0.1:8000 \
  --web-browser-flag=--disable-web-security \
  --web-browser-flag=--user-data-dir="$TMPDIR/autovida-chrome"
```

Este navegador se reserva para el desarrollo local. Para revisar solo la interfaz en un navegador normal se puede utilizar `DEMO=true`.

En el emulador Android habitual:

```
cd src/frontend
flutter run -d IDENTIFICADOR_DEL_EMULADOR \
  --dart-define=API_BASE_URL=http://10.0.2.2:8000
```

En un dispositivo físico, la URL debe ser accesible desde su red. El lanzador escucha en `127.0.0.1`, por lo que esa ejecución local no queda expuesta automáticamente a otros dispositivos. HTTP se permite en debug/profile; la configuración release requiere revisión de red y firma.

1.7. Parada

`Ctrl+C` en la terminal del servicio detiene la API. Flutter se cierra desde su terminal o ventana. Para detener el contenedor:

```
bash ./scripts/levantar_backend.sh --parar
```

La opción no localiza procesos de Flutter o API abiertos en otras terminales. Si el acceso a Docker requiere privilegios, la parada puede necesitar el comando Compose correspondiente con sudo. El volumen se conserva; `down -v` no es una parada normal porque elimina datos.

2. Diagnóstico de errores

2.1. Arranque y herramientas

Síntoma	Comprobación y resolución
No se encuentra `./levantar_backend`	Ejecutar `bash ./scripts/levantar_backend.sh` desde la raíz.
El script rechaza ejecutarse	Utilizar el usuario de desarrollo; no ejecutar todo con sudo.
Docker no responde o no hay Compose	Revisar `docker info`, `docker compose version` y el servicio Docker del equipo.
Falta configuración local	Completar el `.env` que utiliza el lanzador. Un archivo existente vacío no se rellena automáticamente.
Fallan las dependencias Python	Revisar la terminal del script y el entorno `src/backend/.venv`. No confundirlo con una carpeta dentro del paquete.
Flutter no abre	Revisar `flutter doctor -v`, `flutter devices`, entorno gráfico y errores de la terminal.
El script elige Linux pero falla al compilar	Completar CMake, compilador, Ninja, GTK y requisitos de plugins; la detección del lanzador no cubre toda la compilación.
`No pubspec.yaml file found`	Ejecutar Flutter dentro de `src/frontend`.
Resolución de paquetes incompatible	Revisar los mínimos del lock antes de modificar dependencias.
Puerto ocupado	Localizar la ejecución anterior con `ss -ltnp` y detener el proceso identificado o ajustar la ejecución manual.

La ejecución en dos terminales permite separar los errores de Flutter de los del servicio. Las operaciones destructivas sobre datos no son una solución para errores de interfaz o compilación.

Docker y contenedor de datos

Si Docker está instalado pero no responde, comprobar el servicio y arrancarlo si está detenido:

```
sudo systemctl status docker
sudo systemctl start docker
docker info
docker compose version
```

Si `docker info` falla por permisos, el lanzador intenta utilizar `sudo docker`. No debe ejecutarse todo el script con sudo. La opción `-parar` no realiza esa detección de permisos; en ese caso, desde la raíz se puede detener el contenedor con:

```
cd src/backend/proyectFastAPI
sudo docker compose down
```

Cuando falla la conexión del lanzador, revisar el contenedor y sus últimas líneas. Si Docker requiere privilegios, anteponer sudo también a estas consultas:

```
docker ps -a
docker logs --tail 50 autovida_postgres
```

Mensaje o situación	Resolución
`password authentication failed`	Comprobar que el `.env` utiliza las credenciales del volumen existente. Cambiar solo ese archivo no cambia la contraseña ya almacenada.
La base indicada no existe	Confirmar la configuración y que se está usando el volumen previsto; coordinar la recuperación con el equipo.
Contenedor reiniciándose	Revisar el error de sus logs antes de volver a ejecutar el lanzador.
Conexión rechazada o timeout	Comprobar contenedor, puerto `5432` y posibles conflictos de red/VPN.
Error de versión del volumen	Utilizar la versión compatible con sus datos o acordar su migración con el equipo.

El lanzador puede sugerir `docker compose down -v` en algunos de estos casos. Esa orden elimina los datos del volumen; no debe utilizarse para intentar reparar la conexión conservando la información existente. Para la parada habitual se usa `-parar` o `docker compose down`.

Configuración y dependencias

Si aparece `Falta DB_PASSWORD`, editar `src/backend/proyectFastAPI/.env` y completar la clave acordada. Pasarla como variable al comando solo rellena el archivo cuando todavía no existe. Un error de sintaxis al leerlo se resuelve revisando las asignaciones y sus comillas.

Si la API muestra `Define DB_PASSWORD en el entorno antes de iniciar la API`, comprobar que se está usando la versión actual de `scripts/levantar_backend.sh`. Las versiones anteriores validaban el archivo en subprocesos, pero no exportaban sus variables al arrancar Uvicorn. Con el lanzador actual basta con completar el `.env` y ejecutar `bashscripts/levantar_backend.sh`.

Si falla la creación del entorno Python, instalar `python3-venv` para el intérprete utilizado y comprobar su versión. Si falla la instalación de paquetes, revisar el primer error de `pip`, la conexión y los permisos de `src/backend/.venv`. Un entorno incompleto puede apartarse y recrearse con el lanzador; deben conservarse el `.env` y los datos. No se necesita activar manualmente el entorno: el script invoca su intérprete directamente.

Si `flutter` no se encuentra, añadir el directorio `bin` del SDK al `PATH` de la terminal y comprobar `flutter --version`. Si falla `flutter pub get` por versión de Dart o Flutter, utilizar un SDK compatible con `pubspec.lock`; borrar el lock no corrige la incompatibilidad del entorno.

Para errores de compilación Linux, revisar primero `flutter doctor -v` y los paquetes de la sección de requisitos. Después, desde `src/frontend`, ejecutar `flutter pub get` y repetir `flutter run -d linux`. Si el equipo no tiene sesión gráfica, se necesita un destino disponible en `flutter devices`.

Para un puerto ocupado, `ss -ltnp` permite identificar qué proceso escucha en `8000`, `5174` o `5432`. Cerrar la ejecución anterior desde su terminal. El lanzador tiene puertos fijos; no acepta un argumento para cambiarlos. Una segunda ejecución simultánea puede entrar en conflicto con la primera.

2.2. Conexión HTTP

Comprobación básica desde la terminal:

```
curl --fail http://127.0.0.1:8000/
```

Si responde desde terminal pero falla en Chrome por origen, se trata de CORS. Puede utilizarse escritorio o el navegador aislado de desarrollo del lanzador. Abrir la URL en otro navegador no hereda sus opciones.

En un móvil, `127.0.0.1` apunta al dispositivo. Hay que verificar la URL compilada, la accesibilidad de red y la dirección donde escucha el servicio.

Una API en `500` o inaccesible se diagnostica en su terminal y en los mensajes del lanzador. No se corrige activando demo como respuesta automática al fallo. La inicialización de datos y credenciales del servicio debe conservar la configuración existente; volver a lanzar el script no restablece cuentas ya creadas.

2.3. Respuestas rechazadas

Respuesta	Revisión desde el cliente
Credenciales incorrectas	Comprobar usuario, contraseña y rol elegido.
Código 1002	Volver a autenticar; el cliente limpia la sesión al recibirlo.
HTTP 403	Comprobar el perfil y las asociaciones de la operación consumida.
HTTP 409	Revisar duplicados o conflicto de matrícula antes de repetir la acción.
HTTP 422	Comparar nombres, tipos y campos del cuerpo con el contrato utilizado.
HTTP 413 o error de imagen	Revisar tamaño y formato del archivo seleccionado.
HTTP 500	Consultar el error del servicio; el cliente presenta un mensaje general.

Los rechazos de permisos se mantienen como errores. No se sustituyen por datos almacenados ni por respuestas de demostración.

2.4. Información antigua o acciones ausentes

Situación	Punto de revisión
Total de alumnos antiguo	Confirmación de la operación, `classroomsChanged()` y clave de `LoadPanel`.
Foto antigua	Resultado de la subida y `photoChanged()`.
Búsqueda muestra una página incorrecta	Filtrar antes de paginar y reiniciar la clave/índice al cambiar el criterio.
Preferencias no aplicadas	`profile()`, copia de ajustes, guardado de diferencias y `refreshProfile()`.
Agenda vacía	Fecha seleccionada, filtro y destinatario. Las tareas de prueba conservan la fecha de creación.
Falta el acceso a ajustes del alumno	Permiso `profile_mod`.
Falta creación de tarea propia	Permiso `task_mod`.
Falta el botón de completar	Capacidad `canTrackProgress` de la conexión.
No aparecen controles de alta para personal	Comportamiento previsto: solo gestiona alumnos existentes.
Personal de aula incompleto	`staffComplete` y aviso de información confirmada durante la sesión, especialmente en aulas vacías.

Hot reload conserva estado. Si se modifican inicializaciones, debe utilizarse hot restart y autenticar de nuevo. Después de cambiar dependencias o configuración nativa se debe detener y volver a ejecutar Flutter; `flutter clean` se reserva para artefactos de compilación que necesiten regenerarse.

3. Enlaces a código (GIT) y JIRA

3.1. Repositorio de código

- **GitHub:** Repositorio del proyecto AUTOVIDA.

3.2. Gestión del proyecto en JIRA

- **JIRA:** Espacio de Trabajo JIRA CLENCH

4. Actas de Equipo

Las siguientes actas recogen los principales acuerdos y compromisos de las reuniones de CLENCH Software Development entre el 29 de octubre y el 11 de noviembre de 2026. Se presentan de forma resumida y homogénea como registro del trabajo realizado para el proyecto AUTOVIDA: el sistema accesible de agenda y asignación de tareas destinado al C.E.E. Fundación Purísima Concepción.

4.1. Acta 07: Diseño y arquitectura

29 de octubre de 2026

Datos de la reunión

Proyecto: Agenda Accesible — Proyecto AUTOVIDA (C.E.E. Fundación Purísima Concepción)
Asignaturas: Dirección y Gestión de Proyectos (DGP) y Metodologías de Desarrollo Ágil (MDA) — UGR
Horario y modalidad: 08:30–14:30, presencial en el aula de la ETSIIT
Moderador: Pablo de la Torre Roldán **Catalogador:** Miguel Ángel Luque Gómez

4.1.1. Asistencia

Integrante / asistente	Rol	Asistencia
Miguel Ángel Luque Gómez	Catalogador	✓
Yeray Rodríguez Navas	Gestor de Calidad	✓
Julián Carrión Tovar	Gestor de Usabilidad y Accesibilidad	✓
Pablo Hernández Ibáñez	Presentador	✓
Pablo de la Torre Roldán	Moderador	✓
José Rodríguez Fernández	Coordinador	✓
Amine Azzammouri	Catalogador (refuerzo)	✓

4.1.2. Orden del día y desarrollo

1. Iniciar el desarrollo de frontend y backend de acuerdo con el reparto establecido.
2. Diseñar la interfaz teniendo en cuenta los requisitos de accesibilidad.
3. Realizar el modelo relacional de la base de datos.
4. Crear el guion de instalación y adaptar el entorno de trabajo.

Se identificó la necesidad de revisar los requisitos para incluir funcionalidades no contempladas inicialmente, entre ellas las tareas relacionadas con el comedor y la posibilidad de que el alumnado elija avatares. También se trató el tratamiento de imágenes para incorporarlas a las tareas y sus pasos.

4.1.3. Acuerdos

- **DEC-07.1:** Yeray Rodríguez Navas redactará la guía de instalación y adaptación del entorno, con apoyo de Amine Azzammouri.
- **DEC-07.2:** Miguel Ángel Luque Gómez y Pablo de la Torre Roldán diseñarán la versión inicial del modelo relacional de la base de datos.
- **DEC-07.3:** El resto del equipo elaborará los mockups y wireframes del frontend.

4.1.4. Compromisos

ID	Tarea	Responsable(s)	Fecha límite	Prioridad	Estado
T-07.1	Redactar la guía de instalación y adaptación del entorno de trabajo.	Yeray Rodríguez, con apoyo de Amine Azzammouri	05/11/2026	Alta	Pendiente
T-07.2	Diseñar la versión inicial del modelo relacional de la base de datos.	Miguel Ángel Luque y Pablo de la Torre	05/11/2026	Alta	Pendiente
T-07.3	Elaborar mockups y wireframes accesibles del frontend.	Julián Carrión, Pablo Hernández y José Rodríguez	05/11/2026	Media	Pendiente

4.1.5. Próxima convocatoria

Se previó una reunión para el 5 de noviembre de 2026 a las 08:30, para continuar el desarrollo de frontend y backend y revisar los compromisos de diseño y preparación del entorno.

4.2. Acta 08: Revisión de documentación

2 de noviembre de 2026

Datos de la reunión

Proyecto: Agenda Accesible — Proyecto AUTOVIDA (C.E.E. Fundación Purísima Concepción)
Asignaturas: Dirección y Gestión de Proyectos (DGP) y Metodologías de Desarrollo Ágil (MDA) — UGR
Horario y modalidad: 19:00–21:30, telemática a través de la plataforma Discord
Moderador: Pablo de la Torre Roldán **Catalogador:** Miguel Ángel Luque Gómez

4.2.1. Asistencia

Integrante / asistente	Rol	Asistencia
Miguel Ángel Luque Gómez	Catalogador	✓
Yeray Rodríguez Navas	Gestor de Calidad	✓
Julián Carrión Tovar	Gestor de Usabilidad y Accesibilidad	✓
Pablo Hernández Ibáñez	Presentador	✓
Pablo de la Torre Roldán	Moderador	✓
José Rodríguez Fernández	Coordinador	✓
Amine Azzammouri	Catalogador (refuerzo)	✓

4.2.2. Orden del día y desarrollo

1. Realizar cambios en el manual de coordinación y propuesta técnica incorporando las mejoras propuestas por la profesora.
2. Incorporar actas de equipo y cliente en el manual de coordinación.

Se revisó cómo orientar los requisitos y objetivos de la propuesta técnica hacia las necesidades del cliente. También se planificó la incorporación de las actas de equipo y cliente en la estructura del manual de coordinación y se asignó la elaboración del diagrama de arquitectura como parte de la documentación técnica.

4.2.3. Acuerdos

- **DEC-08.1:** De forma conjunta se refinarán los requisitos no funcionales propuestos.
- **DEC-08.2:** De forma conjunta se adaptarán los requisitos para enfocarlos al cliente.
- **DEC-08.3:** De forma conjunta se redefinirán los objetivos de la propuesta técnica para orientarlos al cliente.
- **DEC-08.4:** José Rodríguez - Inclusión de actas de equipo y de cliente en el manual de coordinación.
- **DEC-08.5:** Pablo de la Torre Roldán elaborará e integrará el diagrama de arquitectura en la documentación técnica.

4.2.4. Compromisos

ID	Tarea	Responsable(s)	Fecha límite	Prioridad	Estado
T-08.1	Requisitos no funcionales: El equipo revisará conjuntamente los requisitos no funcionales propuestos y concretará para cada uno criterios verificables de calidad y aceptación. Entregable: versión revisada de los requisitos no funcionales.	Todo el equipo	02/11/2026	Alta	Finalizado
T-08.2	Enfoque al cliente: El equipo reformulará conjuntamente los requisitos para expresarlos desde las necesidades y prioridades del cliente, comprobando que sean claros y validables. Entregable: catálogo de requisitos adaptado al cliente.	Todo el equipo	02/11/2026	Alta	Finalizado
T-08.3	Objetivos de la propuesta técnica: El equipo revisará los objetivos de la propuesta técnica para vincularlos explícitamente con las necesidades del cliente y los requisitos acordados. Entregable: objetivos técnicos actualizados.	Todo el equipo	02/11/2026	Alta	Finalizado
T-08.4	Actas en el manual de coordinación: José Rodríguez incorporará al manual las pautas y plantillas para elaborar y conservar las actas de equipo y cliente. Entregable: manual de coordinación actualizado.	José Rodríguez	02/11/2026	Alta	Finalizado
T-08.5	Diagrama de arquitectura: Pablo de la Torre Roldán elaborará el diagrama de arquitectura de la solución y lo integrará en la documentación técnica. Entregable: diagrama revisado e incorporado a la propuesta.	Pablo de la Torre Roldán	02/11/2026	Alta	Finalizado

4.2.5. Próxima convocatoria

Se previó una reunión para el 5 de noviembre de 2026 a las 08:30, para continuar el desarrollo de frontend y backend y revisar los compromisos de diseño y preparación del entorno.

4.3. Acta 09: Desarrollo e integración de frontend y backend

6 de noviembre de 2026

Datos de la reunión

Proyecto: Agenda Accesible — Proyecto AUTOVIDA (C.E.E. Fundación Purísima Concepción)
Asignaturas: Dirección y Gestión de Proyectos (DGP) y Metodologías de Desarrollo Ágil (MDA) — UGR
Horario y modalidad: 16:00–20:30, presencial en el Piso Estudiante
Tiempo de reunión: aproximadamente 1 hora, incluyendo la puesta en común entre backend y frontend
Tiempo de trabajo: aproximadamente 4 horas y 30 minutos
Moderador: Pablo de la Torre Roldán **Catalogador:** Miguel Ángel Luque Gómez

4.3.1. Asistencia

Integrante / asistente	Rol	Asistencia
José Rodríguez Fernández	Coordinador	✓
Miguel Ángel Luque Gómez	Catalogador	Ausencia justificada
Yeray Rodríguez Navas	Gestor de Calidad	✓
Julián Carrión Tovar	Gestor de Usabilidad y Accesibilidad	Ausencia justificada
Pablo de la Torre Roldán	Moderador	✓
Pablo Hernández Ibáñez	Presentador	✓
Amine Azzammouri	Refuerzo	Ausencia justificada

4.3.2. Orden del día y desarrollo

1. Aclarar el funcionamiento e integración del sistema entre frontend y backend.
2. Actualizar el tablero de Jira tras la incorporación de nuevas historias de usuario.
3. Implementar historias de usuario del backend, incluida la desasignación de alumnos de tutores o personal.
4. Avanzar en los bocetos de la iteración e incorporar el flujo del perfil de alumno.
5. Revisar y corregir el trabajo realizado.

Se revisaron las diferencias entre los datos que necesitaban el frontend y el backend, así como las peticiones del profesorado. Se decidió incorporar aulas y relaciones con alumnos y personal, además de imágenes para tareas, pasos y perfiles. La propuesta inicial de usar avatares en el perfil del alumno se sustituyó por fotografías de los propios alumnos.

También se trató la organización y priorización en Jira de las nuevas historias de usuario. El equipo de frontend revisó la distribución de algunas interfaces y el flujo entre ellas. Durante el trabajo de backend surgieron problemas de versiones e implementación que requirieron depuración y consumieron tiempo adicional; al final se revisó el avance de la sesión.

4.3.3. Acuerdos

- **DEC-09.1:** Añadir al backend la entidad de aulas y sus relaciones con alumnos y profesores, e incorporar imágenes en tareas, pasos y perfiles de alumno.

- **DEC-09.2:** El gestor de calidad preparará las nuevas historias de usuario relacionadas con las aulas.
- **DEC-09.3:** Actualizar los bocetos de Figma para mejorar su accesibilidad y trasladar a Figma los bocetos realizados en papel.
- **DEC-09.4:** Ante los problemas de versiones del backend, volver a una versión previa del código y preparar una nueva versión depurada a partir de ella.

4.3.4. Compromisos

ID	Tarea	Responsable(s)	Fecha límite	Estado
T-09.1	Añadir aulas e imágenes al backend.	Pablo de la Torre Roldán y Yeray Rodríguez Navas	No aplica	Realizado
T-09.2	Actualizar el tablero de Jira con las nuevas historias de usuario.	Yeray Rodríguez Navas	No aplica	Realizado
T-09.3	Completar los bocetos pendientes del frontend.	Pablo Hernández Ibáñez y José Rodríguez Fernández	08/11/2026	Pendiente
T-09.4	Actualizar en Figma los bocetos realizados a mano.	Pablo Hernández Ibáñez, José Rodríguez Fernández y Julián Carrión Tovar	08/11/2026	Pendiente
T-09.5	Desarrollar el frontend de la primera iteración.	Pablo Hernández Ibáñez, José Rodríguez Fernández y Julián Carrión Tovar	11/11/2026	Pendiente
T-09.6	Desarrollar el backend de la primera iteración.	Pablo de la Torre Roldán, Yeray Rodríguez Navas, Miguel Ángel Luque Gómez y Amine Azzammouri	11/11/2026	Pendiente

4.3.5. Próxima convocatoria

Se previó una reunión presencial para el 9 de noviembre de 2026 a las 16:00, aproximadamente, para continuar el desarrollo de la iteración.

4.3.6. Anotaciones

Algunas tareas se realizarían de forma individual por distintos miembros del equipo y no se incluirían en el acta, salvo en el recuento particular de horas.

4.4. Acta 10: Seguimiento del desarrollo y cierre de la iteración 1

10 de noviembre de 2026

Datos de la reunión

Proyecto: Agenda Accesible — Proyecto AUTOVIDA (C.E.E. Fundación Purísima Concepción)
Asignaturas: Dirección y Gestión de Proyectos (DGP) y Metodologías de Desarrollo Ágil (MDA) — UGR
Horario y modalidad: 16:00–21:00, presencial en el Piso Estudiante
Tiempo de reunión: aproximadamente 30 minutos
Tiempo de trabajo: aproximadamente 4 horas y 30 minutos
Moderador: Pablo de la Torre Roldán **Catalogador:** Miguel Ángel Luque Gómez

4.4.1. Asistencia

Integrante / asistente	Rol	Asistencia
José Rodríguez Fernández	Coordinador	✓
Miguel Ángel Luque Gómez	Catalogador	✓
Yeray Rodríguez Navas	Gestor de Calidad	✓
Julián Carrión Tovar	Gestor de Usabilidad y Accesibilidad	✓
Pablo de la Torre Roldán	Moderador	✓
Pablo Hernández Ibáñez	Presentador	✓
Amine Azzammouri	Refuerzo	✓

4.4.2. Orden del día y desarrollo

1. Poner en común los avances de backend y frontend.
2. Finalizar la documentación correspondiente a la Iteración 1.

Los equipos de frontend y backend presentaron el estado del proyecto y las guías y documentos internos preparados para facilitar la comprensión del desarrollo y la coordinación entre módulos. Se revisaron el código, los puntos de integración entre cliente y servidor y la necesidad de unificar terminología, datos y navegación antes de la siguiente iteración.

También se debatieron los últimos ajustes de implementación del frontend, la consistencia visual de sus componentes y la resolución de errores funcionales, procurando mantener la correspondencia con el prototipo y los requisitos. Se revisaron los ajustes de accesibilidad para el alumnado, incluidos el tamaño de fuente, el contraste, la disposición de botones, la lectura por voz y los apoyos visuales. Se acordó comprobar su aplicación en los perfiles de estudiante mediante un caso de uso real.

Por último, se trató la incorporación del manual de calidad al documento del equipo como referencia para revisar los requisitos funcionales y las pautas de accesibilidad, mantenibilidad y trazabilidad.

4.4.3. Acuerdos

- **DEC-10.1:** Revisar el progreso de frontend y backend y cerrar los últimos cambios pendientes para preparar la base técnica de la siguiente iteración.
- **DEC-10.2:** Continuar revisando los ajustes de accesibilidad del alumnado, priorizando la personalización visual, la legibilidad y la claridad del flujo de tareas.

- **DEC-10.3:** Incorporar el manual de calidad como documento de referencia y criterio de revisión durante la Iteración 2.
- **DEC-10.4:** Iniciar de forma organizada la Iteración 2 y validar el alcance de las tareas pendientes antes de la próxima reunión.

4.4.4. Compromisos

ID	Tarea	Responsable(s)	Fecha límite	Estado
T-10.1	Cerrar los ajustes de interfaz y revisar el flujo de uso del frontend.	Pablo Hernández, José Rodríguez y Julián Carrión	12/11/2026	Pendiente
T-10.2	Revisar y ajustar el backend, validar los datos y comprobar la integración con el frontend.	Yeray Rodríguez, Pablo de la Torre, Miguel Ángel Luque y Amine Azzammouri	12/11/2026	Pendiente
T-10.3	Incorporar el manual de calidad al documento del equipo y revisar sus criterios de aplicación.	Yeray Rodríguez y Julián Carrión	12/11/2026	Pendiente
T-10.4	Preparar y redactar la documentación necesaria para la primera entrega.	Todo el equipo	12/11/2026	Pendiente

4.4.5. Próxima convocatoria

Se previó una reunión para el 12 de noviembre de 2026 a las 08:30, aproximadamente, con el objetivo de iniciar la Iteración 2, avanzar en el desarrollo incremental, revisar la accesibilidad y cerrar los pendientes de la iteración anterior.

4.5. Acta 11: Revisión final de la primera iteración

11 de noviembre de 2026

Datos de la reunión

Proyecto: Agenda Accesible — Proyecto AUTOVIDA (C.E.E. Fundación Purísima Concepción)
Asignaturas: Dirección y Gestión de Proyectos (DGP) y Metodologías de Desarrollo Ágil (MDA) — UGR
Horario y modalidad: 10:30–15:00, telemática a través de Discord
Tiempo de reunión: aproximadamente 30 minutos
Tiempo de trabajo: aproximadamente 4 horas y 30 minutos
Moderador: Pablo de la Torre Roldán **Catalogador:** Miguel Ángel Luque Gómez

4.5.1. Asistencia

Integrante / asistente	Rol	Asistencia
José Rodríguez Fernández	Coordinador	✓
Miguel Ángel Luque Gómez	Catalogador	✓
Yeray Rodríguez Navas	Gestor de Calidad	✓
Julián Carrión Tovar	Gestor de Usabilidad y Accesibilidad	✓
Pablo de la Torre Roldán	Moderador	✓
Pablo Hernández Ibáñez	Presentador	✓
Amine Azzammouri	Refuerzo	✓

4.5.2. Orden del día y desarrollo

1. Revisar el prototipo final previsto para la presentación al cliente.
2. Elaborar las guías de frontend y de puesta en marcha del programa.
3. Revisar y pulir la guía de backend.
4. Actualizar el tablero de Jira.
5. Revisar la documentación de cliente y de equipo.
6. Realizar la revisión final de la primera iteración.

Se revisó el prototipo final y se validaron sus flujos principales antes de presentarlo al cliente. El equipo de frontend preparó sus guías técnicas y la guía para probar el programa en local; también se revisó la guía de backend, aclarando la descripción de los endpoints, el esquema de la base de datos y la configuración del servidor.

Se actualizó el tablero Kanban de Jira y se marcaron como terminadas las tareas completadas durante las sesiones de trabajo. Además, se repasó la documentación del sprint, diferenciando los documentos internos de equipo de los materiales dirigidos al cliente, y se unificaron formatos y detalles finales. La retrospectiva permitió valorar el rendimiento, los bloqueos, el cumplimiento de los tiempos y las prácticas que conviene mantener en el siguiente ciclo.

4.5.3. Acuerdos

- **DEC-11.1:** Aprobar por unanimidad el prototipo final y reservar para la planificación de la Iteración 2 algunos cambios propuestos.

- **DEC-11.2:** Incorporar las guías de frontend, backend y puesta en marcha a la documentación oficial del proyecto y actualizarlas ante futuros cambios arquitectónicos.
- **DEC-11.3:** Cerrar en Jira la Iteración 1 y aprobar los entregables asociados.
- **DEC-11.4:** Mantener en futuras retrospectivas el formato de revisión conjunta para detectar pronto desviaciones en la planificación.

4.5.4. Compromisos

ID	Tarea	Responsable(s)	Fecha límite	Estado
T-11.1	Preparar el entorno, el guion y los materiales gráficos para presentar el prototipo al cliente.	Pablo Hernández Ibáñez y José Rodríguez Fernández	12/11/2026	Completado
T-11.2	Subir, dar formato y organizar las guías de frontend, backend y puesta en marcha en el repositorio documental.	Todo el equipo de desarrollo	12/11/2026	Completado
T-11.3	Revisar la ortotipografía y empaquetar la documentación de cliente y equipo para entregar la Iteración 1.	Miguel Ángel Luque y Amine Azzammouri	12/11/2026	Completado
T-11.4	Crear y asignar en Jira las nuevas épicas, historias de usuario y tareas técnicas para iniciar la Iteración 2.	José Rodríguez Fernández	12/11/2026	Pendiente

4.5.5. Próxima convocatoria

Se previó una reunión para el 12 de noviembre de 2026 a las 08:30, aproximadamente, para iniciar la Iteración 2, avanzar en el desarrollo incremental, revisar la accesibilidad y cerrar los pendientes de la iteración anterior.

5. Registro de horas del equipo

Las horas correspondientes a la semana 09/11/2026–11/11/2026 quedan pendientes de registro. Los totales reflejan únicamente las horas registradas hasta el 05/11/2026.

Datos del registro

Integrante: Miguel Ángel Luque Gómez
Periodo: desde 22/09/2026 hasta 11/11/2026
Rol: Catalogador

Semana	Horas trabajadas
22/09/2026–27/09/2026	4
28/09/2026–04/10/2026	4
05/10/2026–11/10/2026	5
12/10/2026–18/10/2026	4
19/10/2026–25/10/2026	6
26/10/2026–01/11/2026	4
02/11/2026–05/11/2026	7
06/11/2026–11/11/2026	8
Total de horas registradas:	42

Datos del registro

Integrante: Yeray Rodríguez Navas
Periodo: desde 22/09/2026 hasta 11/11/2026
Rol: Gestor de Calidad

Semana	Horas trabajadas
22/09/2026–27/09/2026	4
28/09/2026–04/10/2026	4
05/10/2026–11/10/2026	6
12/10/2026–18/10/2026	6
19/10/2026–25/10/2026	7.5
26/10/2026–01/11/2026	4.5
02/11/2026–05/11/2026	7
06/11/2026–11/11/2026	19.5
Total de horas registradas:	58.5

Datos del registro

Integrante: Pablo Hernández Ibáñez
Periodo: desde 22/09/2026 hasta 11/11/2026
Rol: Presentador

Semana	Horas trabajadas
22/09/2026–27/09/2026	5
28/09/2026–04/10/2026	4
05/10/2026–11/10/2026	5
12/10/2026–18/10/2026	6
19/10/2026–25/10/2026	6
26/10/2026–01/11/2026	7,5
02/11/2026–05/11/2026	12,5
06/11/2026–11/11/2026	14
Total de horas registradas:	60

Datos del registro

Integrante: Julian Carrion Tovar
Periodo: desde 22/09/2026 hasta 11/11/2026
Rol: Catalogador

Semana	Horas trabajadas aprox.
22/09/2026–27/09/2026	3,5
28/09/2026–04/10/2026	4
05/10/2026–11/10/2026	4
12/10/2026–18/10/2026	4
19/10/2026–25/10/2026	4
26/10/2026–01/11/2026	4
02/11/2026–05/11/2026	8
06/11/2026–11/11/2026	12,5
Total de horas registradas:	44

Datos del registro

Integrante: José Rodríguez Fernández
Periodo: desde 22/09/2026 hasta 11/11/2026
Rol: Coordinador

Semana	Horas trabajadas
22/09/2026–27/09/2026	2
28/09/2026–04/10/2026	3
05/10/2026–11/10/2026	2
12/10/2026–18/10/2026	4
19/10/2026–25/10/2026	3
26/10/2026–01/11/2026	5
02/11/2026–05/11/2026	7
06/11/2026–11/11/2026	15.5
Total de horas registradas:	41.5

Datos del registro

Integrante: Pablo de la Torre Roldán
Periodo: desde 22/09/2026 hasta 11/11/2026
Rol: Moderador

Semana	Horas trabajadas
22/09/2026–27/09/2026	4
28/09/2026–04/10/2026	4
05/10/2026–11/10/2026	5,5
12/10/2026–18/10/2026	5
19/10/2026–25/10/2026	7,5
26/10/2026–01/11/2026	4
02/11/2026–05/11/2026	7
06/11/2026–11/11/2026	20
Total de horas registradas:	57

Datos del registro

Integrante: Amine
Periodo: desde 22/09/2026 hasta 11/11/2026
Rol: Catalogador

Semana	Horas trabajadas
22/09/2026–27/09/2026	2
28/09/2026–04/10/2026	2
05/10/2026–11/10/2026	3
12/10/2026–18/10/2026	3
19/10/2026–25/10/2026	4,5
26/10/2026–01/11/2026	3,5
02/11/2026–05/11/2026	4
06/11/2026–11/11/2026	14
Total de horas registradas:	36